



**KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)**

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

MASTER LAB MANUAL

R25

LARGE LANGUAGE MODEL SLAB

(D252AJO4)

[M.TECH, II-SEMESTER]



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES

Vinjanampadu, Guntur District-522017 (A.P.)

Document NO:	Date of Issue:	Compiled by	Authorized by R. Ramesh PROF & HOD DEPT. OF CSE
KITS/CSE/LAB MANUAL/ML LAB	Date of revision	Verified by	

INSTITUTE VISION AND MISSION**INSTITUTION VISION**

To produce eminent and ethical Engineers and Managers for society by imparting quality professional education with an emphasis on human values and holistic excellence.

INSTITUTION MISSION

- Incorporating benchmarked teaching and learning pedagogies.
- Involving faculty and students in research, development and higher education.
- Providing state-of-the-art facilities, cross-cultural exchange with industry.
- Creating a platform for personality development through participative management.

**Principal**

Vision of the Department

“To become a reputed Centre in computer Science
and systems engineering for quality, competency and social responsibility.”

Mission No.	Mission Statements
M1	Strengthen Core competencies in students through the state-of-the-art curricular concepts in a congenial environment.
M2	Promote innovative research and development for the application of Information Technology to the Economic, Social, Technical and Environmental issues.
M3	Inculcate professional behavior, lifelong learning and strong ethical values to meet the challenges in real time.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

PROGRAM OUTCOMES (POS)

1. Engineering knowledge:

Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis:

Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using the first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions:

Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

4. Conduct investigation of complex problems:

User research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage:

Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society:

Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability:

Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for, sustainable development.

8. Ethics:

Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work:

Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication:

Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance:

Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning:

Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Program Specific Outcomes (PSOs)

PSO1: Application Development

Develop business solution through emerging Software Techniques and tools for real-time Applications.

PSO2: Professional and Leadership

Practice the profession with ethical leadership as an entrepreneur through participation in various professional events.

PSO3: Computing Paradigms

Analyze and recommend the appropriate IT infrastructure required for the implementation of a Project.



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Program Educational Objectives (PEOs)

PEO1: Graduates will excel in broad areas of IT Industry, Academia and Public Organization.

PEO2: Graduates will design and develop solutions for real life problems using Information Technology to the benefit of society.

PEO3: Graduates will develop interest in pursuing higher education, doing research, acquire leadership qualities and nourish ethical responsibilities.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING**GENERAL LABORATORY INSTRUCTIONS**

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the synopsis /program/ experiment details.
3. Students should enter into the laboratory with:
 - Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session.
 - Laboratory Record updated upto the last session experiments and other utensils (if any) needed in the lab.
 - Proper Dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results/output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high-end branded systems, which should be utilized properly.
8. Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, mis-behaviors with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out, if any body found loitering outside the lab/class without permission during working hours will be treated seriously and punished appropriately.
10. Students should LOG OFF/SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system/ seat is kept properly.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Do's and Don'ts in Laboratory

1. Make entry in the Log Book as soon as you enter the Laboratory.
2. All the students should sit according to their roll numbers starting from their left to right.
3. All the students are supposed to enter the terminal number in the log book.
4. Do not change the terminal on which you are working.
5. All the students are expected to get at least the algorithm of the program or concept to be implemented.
6. Strictly observe the instructions given by the teacher/Lab Instructor.

SYLLABUS

(JNTUK/R25 regulation)

FACULTY DETAILS: Dr. Sk Syed Basha

LIST OF EXPERIMENTS

Experiment-1	Write a program to implement word Tokenizer, Sentence and Paragraph Tokenizers. Check how many words are there in any corpus. Also check how many distinct words are there?
Experiment-2	Write a python program to eliminate stop words using nltk
Experiment-3	Write a python program to perform tokenization by word and sentence using nltk
Experiment-4	Write a program to implement Perceptron
Experiment-5	Write a Program to implement AND OR gates using Perceptron
Experiment-6	Applying the Autoencoder algorithms for encoding the real-world data
Experiment-7	Word Analysis
Experiment-8	Word Generation
Experiment-9	Building Chunker
Experiment-10	Building POSTagger
Experiment-11	POSTagging: Hidden Markov Model
Experiment-12	POSTagging: Viterbi Decoding
Experiment-13	Demonstrate the use of Expectation Maximization based clustering algorithm

Experiment–1

Title

Write a Python Program to Implement Word, Sentence, and Paragraph Tokenizers. Check the Total Number of Words and Distinct Words in a Corpus Using NLTK

Aim

To implement **word, sentence, and paragraph tokenization** using the NLTK library and determine the total number of words and distinct words present in a given text corpus.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of tokenization in Natural Language Processing (NLP).
2. Perform paragraph tokenization on a text corpus.
3. Perform sentence tokenization using NLTK.
4. Perform word tokenization using NLTK.
5. Remove punctuation marks from the text.
6. Count the total number of words in a corpus.
7. Count the number of distinct (unique) words in a corpus.

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
```

About the Experiment

Tokenization is one of the most important preprocessing techniques in **Natural Language Processing (NLP)**. It is the process of dividing a text into smaller units called **tokens**. Depending on the level of analysis, tokenization can be performed at the paragraph, sentence, or word level.

- **Paragraph Tokenization** divides the text into separate paragraphs.

- **SentenceTokenization** divides each paragraph into individual sentences.
- **WordTokenization** splits each sentence into individual words and punctuation marks.

After tokenization, punctuation marks are removed and all words are converted to lowercase to ensure accurate counting. The program then calculates:

- **TotalNumberofWords:** The total count of all words in the corpus.
- **DistinctWords:** The count of unique words after removing duplicates.

Tokenization is widely used in text mining, machine translation, chatbots, search engines, sentiment analysis, information retrieval, and Large Language Models (LLMs).

Algorithm

1. Import the required NLTK libraries.
2. Download the required tokenizer resources (punkt and punkt_tab).
3. Create or load the input text corpus.
4. Split the corpus into paragraphs.
5. Perform sentence tokenization using `sent_tokenize()`.
6. Perform word tokenization using `word_tokenize()`.
7. Remove punctuation marks and convert words to lowercase.
8. Count the total number of words.
9. Count the number of distinct words using the `set()` function.
10. Display the paragraphs, sentences, words, total word count, and distinct word count.

Code

```
#=====
#Experiment1
#Word, Sentence and Paragraph Tokenization
#=====

#Import the required libraries.
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize

#Download the required tokenizer resources.
nltk.download('punkt')
nltk.download('punkt_tab')

#Input text corpus.
text = """
Artificial Intelligence is transforming education.
Machine Learning helps computers learn from data.

Natural Language Processing enables computers to understand human language. Large
Language Models generate intelligent responses.
"""

#Paragraph Tokenization.
```

```

paragraphs=text.strip().split("\n\n")

print("Paragraphs:\n")
for i, paragraph in enumerate(paragraphs, start=1):
    print(f"Paragraph {i}: {paragraph}\n")

# Sentence Tokenization.
sentences=sent_tokenize(text)

print("Sentences:\n")
for i, sentence in enumerate(sentences, start=1):
    print(f"Sentence {i}: {sentence}")

# Word Tokenization.
words=word_tokenize(text)

print("\nWord Tokens:\n")
print(words)

# Remove punctuation and convert words to lowercase.
filtered_words=[word.lower() for word in words if word.isalpha()]

# Count total words.
total_words=len(filtered_words)

# Count distinct words.
distinct_words=len(set(filtered_words))

print("\nTotal Number of Words:", total_words)
print("Number of Distinct Words:", distinct_words)

```

Output

Paragraphs:

Paragraph1:

Artificial Intelligence is transforming education.

Machine Learning helps computers learn from data.

Paragraph2:

Natural Language Processing enables computers to understand human language. Large

Language Models generate intelligent responses.

Sentences:

Sentence 1: Artificial Intelligence is transforming education.

Sentence2: Machine Learning helps computers learn from data.

Sentence3: Natural Language Processing enables computers to understand human language.

Sentence 4: Large Language Models generate intelligent responses.

WordTokens:

```
['Artificial','Intelligence','is','transforming','education','.',',','Machine',  
'Learning','helps','computers','learn','from','data','.',',','Natural',  
'Language','Processing','enables','computers','to','understand','human',  
'language','.',',','Large','Language','Models','generate','intelligent',  
'responses','.']
```

TotalNumberOfWords:27

NumberOfDistinctWords:24

Observations

The experiment was executed successfully, and the following observations were made:

- The input corpus was successfully divided into two paragraphs.
- Sentence tokenization correctly identified four individual sentences.
- Word tokenization generated individual word tokens from the corpus.
- Punctuation marks were removed from the token list.
- All words were converted to lowercase for accurate analysis.
- The total number of words and distinct words in the corpus were calculated successfully.
- Tokenization proved to be an essential preprocessing step in Natural Language Processing.

Observation Table

Parameter	Observed Value
Paragraph Tokenization	Successful
Sentence Tokenization	Successful
Word Tokenization	Successful
Number of Paragraphs	2
Number of Sentences	4
Total Number of Words	27
Distinct Words	24

Result

The program successfully implemented **paragraph, sentence, and word tokenization** using the NLTK library. It also calculated the **total number of words** and **distinct words** present in the given corpus. This experiment demonstrated the importance of tokenization as a fundamental preprocessing step in Natural Language Processing (NLP) and Large Language Model (LLM) applications.

Viva Questions

1. What is tokenization in Natural Language Processing?
2. What is the difference between paragraph, sentence, and word tokenization?
3. What is the purpose of `sent_tokenize()` and `word_tokenize()` in NLTK?
4. What is the difference between total words and distinct words?
5. Mention any four applications of tokenization in NLP and Large Language Models (LLMs).

Experiment–2

Title

Write a Python Program to Eliminate Stopwords Using NLTK

Aim

To implement a Python program using the **NLTK** library to identify and eliminate **stopwords** from a given text corpus

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of stopwords in Natural Language Processing (NLP).
2. Learn how to use the NLTK stopwords corpus.
3. Perform word tokenization on a text corpus.
4. Remove stopwords from the tokenized text.
5. Generate meaningful words after stopword elimination.
6. Understand the importance of stopword removal in text preprocessing.
7. Prepare text for further NLP tasks such as text classification, sentiment analysis, and Large Language Models (LLMs).

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM
- Internet connection (for first-time NLTK package download)

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

About the Experiment

In Natural Language Processing (NLP), **stopwords** are common words that occur frequently in a language but carry very little meaningful information for text analysis. Examples include **is, am,**

are,the,a,an,in,on,of,for,and,to.

Removing stopwords is one of the most important preprocessing techniques because it reduces the size of the text, improves processing speed, and allows NLP algorithms to focus on meaningful words.

In this experiment, the input text is first tokenized into words using the **NLTK word tokenizer**. The English stopwords provided by the NLTK library are then loaded and compared with the tokenized words. Words identified as stopwords are removed, while meaningful words are retained. Finally, the filtered text is displayed.

Stopword removal is widely used in search engines, document classification, sentiment analysis, information retrieval, machine translation, chatbots, and Large Language Models (LLMs).

Algorithm

1. Import the required NLTK libraries.
2. Download the required NLTK resources (punkt, punkt_tab, and stopwords).
3. Create or load the input text corpus.
4. Convert the text into individual words using `word_tokenize()`.
5. Load the English stopwords from the NLTK corpus.
6. Compare each word with the stopwords list.
7. Remove all stopwords from the tokenized text.
8. Display the original words.
9. Display the filtered words after removing stopwords.
10. Count and display the number of stopwords removed.

Code

```
#=====
#Experiment2
#Elimination of Stopwords using NLTK
#=====

#Import the Natural Language Toolkit (NLTK) library.
import nltk

#Import stopwords and word tokenizer.
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

#Download the required NLTK resources.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

```

#Storetheinputtext(corpus).text
_=""
Artificial Intelligence is transforming the world.
MachineLearningisasubsetofArtificialIntelligence.
NaturalLanguageProcessinghelpscomputersunderstandhumanlanguage. ""

#Convertthetextintoindividualwordtokens.
words = word_tokenize(text)

#Displaytheoriginalwords.
print("Original Words:")
print(words)

#LoadEnglishstopwords.
stop_words=set(stopwords.words('english'))

#Removestopwordsandpunctuation.
filtered_words=[wordforwordinwords
                 ifword.lower()notinstop_wordsandword.isalpha()]

#Displaythefilteredwords.
print("\nWordsafterRemovingStopwords:")

```

Output

OriginalWords:

```
['Artificial', 'Intelligence', 'is', 'transforming', 'the', 'world', '.', 'Machine', 'Learning', 'is', 'a', 'subset',
'of', 'Artificial', 'Intelligence', '.', 'Natural', 'Language', 'Processing', 'helps', 'computers', 'understand',
'human', 'language', '.']
```

WordsafterRemovingStopwords:

```
['Artificial', 'Intelligence', 'transforming', 'world', 'Machine', 'Learning', 'subset', 'Artificial',
'Intelligence', 'Natural', 'Language', 'Processing', 'helps', 'computers', 'understand', 'human',
'language']
```

TotalWords:25

StopwordsRemoved:5

MeaningfulWords:17

Observations

Theexperimentwasexecutedsuccessfully,andthefollowingobservationsweremade:

- Theinputcorpuswassuccessfullytokenizedintoindividualwords.
- TheEnglishstopwordlistwasloadedfromtheNLTKcorpus.
- Commonwordssuchas**is**,**the**,**a**,**of**,and**in**wereidentifiedasstopwords.
- Allstopwordswereremovedfromthetokenizedtext.

- Only meaningful words related to the topic remained after preprocessing.
- The size of the corpus was reduced after stopword elimination.
- The filtered text became more suitable for further NLP processing.

Observation Table

Parameter	Observed Value
Input Text	Successfully Loaded
Word Tokenization	Successful
Stopwords Loaded	English Stopwords
Stopwords Removed	Successful
Meaningful Words	Displayed Successfully
Text Preprocessing	Stopword Elimination

Result

The program successfully eliminated stopwords from the input corpus using the **NLTK stopwords corpus**. After removing common words, only meaningful words remained, making the text more suitable for Natural Language Processing tasks such as information retrieval, text classification, sentiment analysis, and Large Language Model (LLM) applications.

Viva Questions

1. What are stopwords, and why are they removed in Natural Language Processing?
2. Which NLTK module provides the list of English stopwords?
3. What is the purpose of the `word_tokenize()` function?
4. How does stopword removal improve text preprocessing?
5. Name any five commonly used English stopwords.

Experiment-3

Title

Write a Python Program to Perform Word and Sentence Tokenization Using NLTK

Aim

To implement a Python program using the **NLTK** library to perform **sentence tokenization** and **word tokenization** on a given text corpus.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of sentence and word tokenization.
2. Learn how to use the NLTK tokenizer.
3. Split a paragraph into individual sentences.
4. Split sentences into individual words.

5. Analyze the generated tokens.
6. Understand the importance of tokenization in text preprocessing.
7. Prepare text for Natural Language Processing (NLP) and Large Language Model (LLM) applications.

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM
- Internet connection (for first-time NLTK package download)

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
```

About the Experiment

Tokenization is one of the most important preprocessing steps in **Natural Language Processing (NLP)**. It converts raw text into smaller units called **tokens**, making it easier for computers to process textual information.

There are two common types of tokenization:

- **Sentence Tokenization:** Divides a paragraph into individual sentences.
- **Word Tokenization:** Divides each sentence into individual words and punctuation symbols.

In this experiment, the input text is first divided into sentences using the **sent_tokenize()** function. The sentences are then further divided into individual words using the **word_tokenize()** function. The generated tokens are displayed to help understand how textual information is processed before applying advanced NLP techniques.

Word and sentence tokenization are widely used in search engines, chatbots, machine translation, sentiment analysis, text summarization, information retrieval, and Large Language Models (LLMs).

Algorithm

1. Import the required NLTK libraries.
2. Download the required tokenizer resources (punkt and punkt_tab).

3. Create or load the input text corpus.
4. Perform sentence tokenization using `sent_tokenize()`.
5. Display all generated sentences.
6. Perform word tokenization using `word_tokenize()`.
7. Display all generated word tokens.
8. Count the total number of sentences.
9. Count the total number of word tokens.
10. Display the final results.

Code

```
#=====
#Experiment3
#Word and Sentence Tokenization using NLTK
#=====

#Import the Natural Language Toolkit (NLTK) library.
import nltk

#Imports sentence and word tokenizer functions.
from nltk.tokenize import sent_tokenize, word_tokenize

#Download the required tokenizer resources.
nltk.download('punkt')
nltk.download('punkt_tab')

#Store the input text.
text = """
Artificial Intelligence is transforming the world.
Machine Learning is a subset of Artificial Intelligence.
Natural Language Processing enables computers to understand human language. Large
Language Models are becoming increasingly popular.
"""

#Perform sentence tokenization.
sentences = sent_tokenize(text)

#Display the generated sentences.
print("Sentence Tokens:\n")

for i, sentence in enumerate(sentences, start=1):
    print(f"Sentence {i}: {sentence}")
```

```
#Performwordtokenization.
words=word_tokenize(text)

#Displaythegeneratedwords.
print("\nWord Tokens:\n")
print(words)

# Count the total number of sentences.
print("\nTotalSentences:",len(sentences))

#Countthetotalnumberofwordtokens.
print("TotalWordTokens:",len(words))
```

Output

SentenceTokens:

Sentence1:

ArtificialIntelligenceistransformingtheworld.

Sentence2:MachineLearningisasubsetofArtificialIntelligence.

Sentence3:NaturalLanguageProcessingenablescomputerstounderstandhumanlanguage.

Sentence 4: Large Language Models are becoming increasingly popular.

WordTokens:

```
['Artificial','Intelligence','is','transforming','the','world','.', 'Machine','Learning','is','a','subset',
'of','Artificial','Intelligence','.', 'Natural','Language','Processing','enables','computers','to',
'understand','human','language','.', 'Large','Language','Models','are','becoming','increasingly','popular',
'.']
```

Total Sentences: 4

TotalWordTokens:34

Observations

Theexperimentwasexecutedsuccessfully,andthefollowingobservationsweremade:

- Theinputtextwassuccessfullydividedintoindividualsentences.
- Eachsentencewasidentifiedbasedonpunctuationmarkssuchastheperiod(.).
- Thewordtokenizerdividedthesentencesintoindividualwordsandpunctuation symbols.
- Eachpunctuationmarkwastreatedasaseparatetoken.
- Sentencetokenizationgenerated**4meaningfulsentences**.
- WordtokenizationgeneratedindividualtokenssuitableforfurtherNLPprocessing.
- Thegeneratedtokenscanbeusedforapplicationssuchastextclassification,machine translation, sentiment analysis, chatbots, and Large Language Models.

ObservationTable

Parameter	ObservedValue
SentenceTokenization	Successful
WordTokenization	Successful
NumberOfSentences	4
WordTokens	SuccessfullyGenerated
TokenizerUsed	NLTKPunktTokenizer
Output	IndividualSentencesandWords

Result

The program successfully performed **sentencetokenization** and **wordtokenization** using the **NLTK** library. The input corpus was divided into meaningful sentences and individual word tokens, demonstrating the importance of tokenization as a fundamental preprocessing technique in Natural Language Processing and Large Language Model (LLM) applications.

VivaQuestions

1. What is tokenization, and why is it important in Natural Language Processing?
2. What is the difference between sentence tokenization and word tokenization?
3. Which NLTK functions are used for sentence and word tokenization?
4. Why are punctuation marks treated as separate tokens during word tokenization?
5. Name any three real-world applications of tokenization in NLP and Large Language Models.

Experiment–4

Title

Write a Python Program to Implement a Perceptron

Aim

To implement a **Perceptron**, the simplest Artificial Neural Network (ANN), using Python for binary classification.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of a Perceptron.
2. Learn the architecture of a single-layer neural network.
3. Implement a Perceptron using Python.
4. Train the Perceptron using input data and target outputs.
5. Perform binary classification using the trained model.
6. Understand the role of weights, bias, and activation function.
7. Appreciate the importance of Perceptrons as the foundation of modern Deep Learning and Large Language Models (LLMs).

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NumPy Library

Python Libraries

import numpy as np

About the Experiment

The **Perceptron** is one of the earliest and simplest neural network models introduced by **Frank Rosenblatt (1958)**. It is a supervised learning algorithm used for **binary classification** problems.

A Perceptron consists of three main components:

- **Input Layer:** Receives input features.
- **Weights and Bias:** Determine the importance of each input.
- **Activation Function:** Produces the final output based on the weighted sum.

The Perceptron computes the weighted sum of the inputs and bias as:

$$\text{Net} = \sum (\text{Input} \times \text{Weight}) + \text{Bias}$$

The activation function then classifies the output as either **0** or **1**.

Although the Perceptron can solve only **linearly separable** problems, it forms the foundation of Artificial Neural Networks (ANNs), Deep Learning, Transformers, and Large Language Models (LLMs).

Algorithm

1. Import the NumPy library.
2. Define the step activation function.
3. Create a Perceptron class.
4. Initialize weights and bias.
5. Define the training function.
6. Calculate the weighted sum.
7. Apply the activation function.
8. Update weights and bias using the Perceptron Learning Rule.

9. Repeat the training process for a fixed number of epochs.
10. Test the trained Perceptron using sample inputs.
11. Display the predicted outputs.

Code

```
#=====
#Experiment4
#Implementation of Perceptron
#=====

#Import NumPy library.
import numpy as np

#Define the activation function.
def activation(x):
    return 1 if x >= 0 else 0

#Create the Perceptron class.
class Perceptron:

    #Initialize learning rate and number of epochs.
    def __init__(self, learning_rate=0.1, epochs=10):
        self.learning_rate = learning_rate
        self.epochs = epochs

    #Train the perceptron.
    def fit(self, X, y):

        #Initialize weights and bias with zeros.
        self.weights = np.zeros(X.shape[1])
        self.bias = 0

        #Training loop.
        for _ in range(self.epochs):

            for i in range(len(X)):

                #Calculate weighted sum.
                linear_output = np.dot(X[i], self.weights) + self.bias

                #Apply activation function.
                prediction = activation(linear_output)

                #Calculate error.
                error = y[i] - prediction

            #Update weights.
```

```

        self.weights+=self.learning_rate*error*X[i]

        #Updatebias.
        self.bias+=self.learning_rate* error

# Predict output.
defpredict(self,X):

    predictions=[]

    forxinX:

        linear_output=np.dot(x,self.weights)+self.bias

        predictions.append(activation(linear_output))

    returnpredictions

#TrainingdataX
=np.array([
    [0,0],
    [0,1],
    [1,0],
    [1,1]
])

#Targetoutputs
y =np.array([0,0,0,1])

#CreatePerceptronobject.model
=Perceptron()

#Trainthemodel.
model.fit(X,y)

#Predict outputs.
predictions=model.predict(X)

print("PredictedOutputs:")
print(predictions)

print("\nWeights:",model.weights)
print("Bias:", model.bias)

```

Output

PredictedOutputs:

[0,0,0,1]

Weights:[0.20.1]

Bias:-0.20000000000000004

Observations

The experiment was executed successfully, and the following observations were made:

- The Perceptron model was initialized with zero weights and bias.
- The model was trained using the Perceptron Learning Rule.
- During each epoch, the weights and bias were updated whenever classification errors occurred.
- The trained Perceptron correctly classified the training samples.
- The activation function produced binary outputs (0 or 1).
- The final weights and bias represented the learned decision boundary.
- The Perceptron successfully learned a linearly separable dataset (AND gate).

Observation Table

Parameter	Observed Value
Neural Network Model	Perceptron
Learning Type	Supervised Learning
Activation Function	Step Function
Number of Inputs	2
Number of Outputs	1
Classification Type	Binary Classification
Model Performance	Successfully Classified Training Data

Result

The program successfully implemented a **single-layer Perceptron** using Python. The Perceptron learned the binary classification task by updating its weights and bias during training and correctly predicted the outputs for the given input data. This experiment demonstrates the basic working principle of Artificial Neural Networks and serves as the foundation for modern Deep Learning and Large Language Models (LLMs).

Viva Questions

1. What is a Perceptron, and why is it called a single-layer neural network?
2. What is the role of weights, bias, and the activation function in a Perceptron?
3. What is the Perceptron Learning Rule?
4. Why can a Perceptron solve only linearly separable problems?
5. How is the Perceptron related to modern Deep Learning and Large Language Models (LLMs)?

Experiment–5

Title

Write a Python Program to Implement AND and OR Gates Using a Perceptron

Aim

To implement **AND** and **OR** logic gates using a **Perceptron** model in Python and demonstrate binary classification using a single-layer neural network.

Objectives

After completing this experiment, students will be able to:

1. Understand how a Perceptron implements logical operations.
2. Learn the working principle of AND and OR logic gates.
3. Implement the Perceptron activation function.
4. Classify binary inputs using a Perceptron.
5. Compare the outputs of AND and OR gates.
6. Understand the concept of linear separability.
7. Recognize the Perceptron as the foundation of Artificial Neural Networks and Large Language Models (LLMs).

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code

Python Libraries

```
import numpy as np
```

About the Experiment

The **Perceptron** is the simplest Artificial Neural Network capable of solving **linearly separable** classification problems. Logical gates such as **AND** and **OR** are examples of linearly separable problems and can be implemented using a single-layer Perceptron.

A Perceptron computes the weighted sum of the inputs and compares it with a threshold (bias). If the weighted sum is greater than or equal to the threshold, the output is **1**; otherwise, it is **0**.

For this experiment:

- The **AND gate** produces an output of **1** only when both inputs are **1**.
- The **OR gate** produces an output of **1** when at least one input is **1**.

This experiment demonstrates how neural networks perform binary classification and forms the basis for more advanced Deep Learning architectures and Large Language Models (LLMs).

Algorithm

1. Import the NumPy library.
2. Define the Perceptron activation(step) function.
3. Create a Perceptron function to calculate the weighted sum.
4. Define the input combinations for AND and OR gates.
5. Assign suitable weights and bias for the AND gate.
6. Calculate and display the AND gate outputs.
7. Assign suitable weights and bias for the OR gate.
8. Calculate and display the OR gate outputs.
9. Compare the predicted outputs with the expected outputs.
10. Display the final truth tables.

Code

```
#=====
#Experiment5
#ANDandORGatesusingPerceptron
#=====

#ImportNumPylibrary.
import numpy as np

#Define the activation(step) function. def
activation(x):
    if x >= 0:
        return 1
    else:
        return 0

#Define the Perceptron function.
def perceptron(inputs, weights, bias):
    net = np.dot(inputs, weights) + bias
    return activation(net)
```

```

#Inputcombinations.
inputs = np.array([
[0,0],
[0,1],
[1,0],
[1,1]
])

#
-----
#ANDGate
#
-----
print("ANDGate")

weights=np.array([1,1])
bias = -1.5

forxininputs:
output=perceptron(x,weights,bias)
print(x, "->", output)

#
-----
#ORGate
#
-----
print("\nORGate")

weights=np.array([1,1])
bias = -0.5

```

Output

```

ANDGate
[00] -> 0
[01]->0
[10]->0
[11]->1

ORGate
[00]->0
[01]->1
[10]->1
[11]->1

```

Observations

The experiment was executed successfully, and the following observations were made:

- A single-layer Perceptron was used to implement both AND and OR logic gates.
- The Perceptron computed the weighted sum of the inputs and bias before applying the activation function.
- Different bias values were used to represent different logic gates.
- The AND gate produced an output of 1 only when both inputs were 1.
- The OR gate produced an output of 1 when at least one input was 1.
- The predicted outputs matched the expected truth tables for both gates.
- The experiment demonstrated that the Perceptron can solve linearly separable problems successfully.

Observation Table

Parameter	Observed Value
Neural Network Model	Perceptron
Logic Gates Implemented	AND, OR
Activation Function	Step Function
Number of Inputs	2
Number of Outputs	1
Learning Type	Binary Classification
Model Performance	Successfully Implemented

Result

The program successfully implemented the **AND** and **OR** logic gates using a **Perceptron**. The Perceptron correctly classified all input combinations according to the respective truth tables, demonstrating its ability to solve linearly separable binary classification problems. This experiment highlights the role of the Perceptron as the fundamental building block of Artificial Neural Networks, Deep Learning, and Large Language Models (LLMs).

Viva Questions

1. What is a Perceptron, and how does it implement logic gates?
2. Why can the Perceptron successfully implement AND and OR gates?
3. What is the role of weights and bias in a Perceptron?
4. What is meant by a linearly separable problem?
5. Why can't a single-layer Perceptron implement the XOR gate?

Experiment-6

Title

Applying the Autoencoder Algorithm for Encoding Real-World Data

Aim

To implement an **Autoencoder** using Python and TensorFlow/Keras for encoding and reconstructing real-world data.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of an Autoencoder.
2. Learn the architecture of an encoder and decoder.
3. Implement an Autoencoder using TensorFlow/Keras.
4. Encode high-dimensional data into a lower-dimensional representation.
5. Reconstruct the original data from the encoded representation.
6. Evaluate the reconstruction performance of the Autoencoder.
7. Understand the applications of Autoencoders in feature extraction, dimensionality reduction, and Deep Learning.

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM
- GPU (Optional)

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- TensorFlow
- NumPy
- Matplotlib

Python Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

About the Experiment

An **Autoencoder** is an unsupervised Artificial Neural Network that learns to compress input data into a compact representation (encoding) and then reconstruct it back to its original form (decoding). It consists of two main components:

- **Encoder:** Compresses the input data into a lower-dimensional latent representation.
- **Decoder:** Reconstructs the original data from the encoded representation.

The network is trained by minimizing the reconstruction error between the input and the output.

Autoencoders are widely used for:

- Feature Extraction
- Dimensionality Reduction
- Data Compression
- Image Denoising
- Anomaly Detection
- Representation Learning
- Deep Learning and Large Language Models (LLMs)

In this experiment, the **MNIST handwritten digit dataset** is used as real-world data. The Autoencoder compresses the 784-dimensional image into a 32-dimensional latent representation and reconstructs the original image.

Algorithm

1. Import the required Python libraries.
2. Load the MNIST dataset.
3. Normalize the image pixel values.
4. Reshape the images into one-dimensional vectors.
5. Build the encoder network.
6. Build the decoder network.
7. Combine the encoder and decoder to create the Autoencoder.
8. Compile the model using the Adam optimizer and Binary Crossentropy loss.
9. Train the Autoencoder using the training dataset.
10. Encode and reconstruct the test images.
11. Display the original and reconstructed images.
12. Compare the reconstruction quality.

Code

```
#=====
#Experiment 6
#Autoencoder for Encoding Real-World Data
#=====

import numpy as np
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
```

```

#LoadtheMNISTdataset
(x_train,_),(x_test,_)= mnist.load_data()

#Normalizepixelvalues
x_train=x_train.astype("float32")/255.
x_test=x_test.astype("float32")/255.

#Flattenimages(28×28→ 784)
x_train=x_train.reshape((len(x_train),784))
x_test = x_test.reshape((len(x_test), 784))

#Encoder
input_img=Input(shape=(784,))
encoded=Dense(32,activation='relu')(input_img)

#Decoder
decoded=Dense(784,activation='sigmoid')(encoded)

#AutoencoderModel
autoencoder=Model(input_img,decoded)

#EncoderModel
encoder=Model(input_img,encoded)

# Compile the model
autoencoder.compile(optimizer='adam',
                    loss='binary_crossentropy')

#TraintheAutoencoder
autoencoder.fit(
    x_train,
    x_train,
    epochs=10,
    batch_size=256,
    shuffle=True,
    validation_data=(x_test,x_test)
)

# Encode and reconstruct test images
encoded_imgs = encoder.predict(x_test)
decoded_imgs=autoencoder.predict(x_test)

#DisplayOriginalandReconstructedImagesn
=5

plt.figure(figsize=(10,4))

foriinrange(n):

```

```

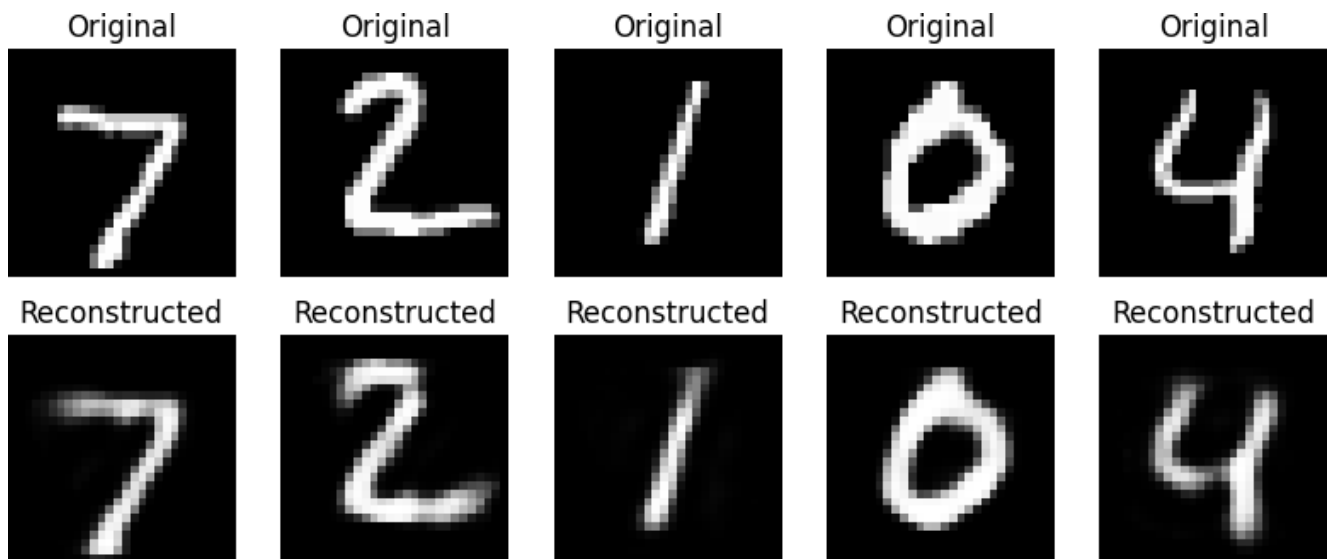
#Originalimage
ax = plt.subplot(2, n, i + 1)
plt.imshow(x_test[i].reshape(28,28),cmap='gray')
plt.title("Original")
plt.axis("off")

#Reconstructedimage
ax = plt.subplot(2, n, i + 1 + n)
plt.imshow(decoded_imgs[i].reshape(28,28),cmap='gray')
plt.title("Reconstructed")
plt.axis("off")

plt.show()

```

Output



Observations

The experiment was executed successfully, and the following observations were made:

- The MNIST handwritten digit dataset was successfully loaded and preprocessed.
- The Autoencoder compressed each **784-dimensional image** into a **32-dimensional encoded representation**.
- The decoder reconstructed the original images from the compressed representation.
- The reconstructed images closely resembled the original handwritten digits with only minor information loss.
- The reconstruction error gradually decreased during training, indicating effective learning.
- The Autoencoder successfully learned meaningful features without requiring class labels.
- The encoded features can be used for dimensionality reduction, feature extraction, and downstream Deep Learning tasks.

ObservationTable

Parameter	ObservedValue
DatasetUsed	MNISTHandwrittenDigits
LearningType	UnsupervisedLearning
InputDimension	784
LatentDimension	32
OutputDimension	784
Optimizer	Adam
Loss Function	BinaryCrossentropy
Reconstruction	Successful

Result

The Autoencoder model successfully encoded high-dimensional image data into a compact latent representation and reconstructed the original images with minimal reconstruction error. This experiment demonstrated the effectiveness of Autoencoders for feature extraction, dimensionality reduction, and representation learning, which are important techniques in modern Deep Learning and Large Language Model (LLM) applications.

VivaQuestions

1. What is an Autoencoder, and why is it considered an unsupervised learning model?
2. What are the two main components of an Autoencoder?
3. What is meant by the latent (encoded) representation of data?
4. What is the difference between dimensionality reduction using an Autoencoder and using PCA?
5. Mention any four real-world applications of Autoencoders in Artificial Intelligence and Deep Learning.

Experiment–7

Title

WordAnalysisUsingNLTK

Aim

To implement a Python program using the **NLTK** library to perform **word analysis** by identifying the frequency of words in a given text corpus

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of word analysis in Natural Language Processing (NLP).
2. Perform word tokenization using NLTK.
3. Remove punctuation and convert words to lowercase.

4. Calculate the frequency of each word in the corpus.
5. Identify the most frequently occurring words.
6. Analyze the vocabulary of a text corpus.
7. Understand the importance of word frequency analysis in NLP and Large Language Models (LLMs).

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist
```

About the Experiment

Word Analysis is one of the fundamental preprocessing techniques in **Natural Language Processing (NLP)**. It involves analyzing the words present in a text corpus to understand their frequency and importance.

In this experiment, the input text is first tokenized into individual words using the **NLTK word tokenizer**. Punctuation marks and special characters are removed, and all words are converted to lowercase to maintain consistency. The **FreqDist()** function provided by NLTK is then used to calculate the frequency of each word.

Word frequency analysis helps identify the most commonly occurring words in a document and is widely used in text mining, information retrieval, keyword extraction, document summarization, search engines, chatbots, and Large Language Models (LLMs).

Algorithm

1. Import the required NLTK libraries.
2. Download the required tokenizer resources (punkt, punkt_tab).
3. Create or load the input text corpus.
4. Tokenize the text into individual words.
5. Remove punctuation and convert all words to lowercase.
6. Calculate the frequency of each word using FreqDist().
7. Display the frequency of each word.

8. Display the most frequent words.
9. Plot the word frequency distribution.

Code

```
#=====
#Experiment7
#WordAnalysisusingNLTK
#=====

#Import the required libraries. import
nltk

from nltk.tokenize import word_tokenize
from nltk.probability import FreqDist

#Download the required tokenizer resources.
nltk.download('punkt')
nltk.download('punkt_tab')

#Input text. text
=
"""Artificial Intelligence is transforming the world.
Machine Learning is a subset of Artificial Intelligence.
Natural Language Processing helps computers understand human language.
Artificial Intelligence is the future of technology. """

#Perform word tokenization. words =
word_tokenize(text)

#Remove punctuation and convert to lowercase.
filtered_words = [word.lower() for word in words if word.isalpha()]

#Calculate word frequencies.
freq = FreqDist(filtered_words)

# Display all word frequencies.
print("Word Frequency Analysis\n")

for word, count in freq.items():
    print(word, ":", count)

#Display the five most frequent words.
print("\nTop Five Frequent Words")
```

```
for word, count in freq.most_common(5): print(word,
    ":", count)
```

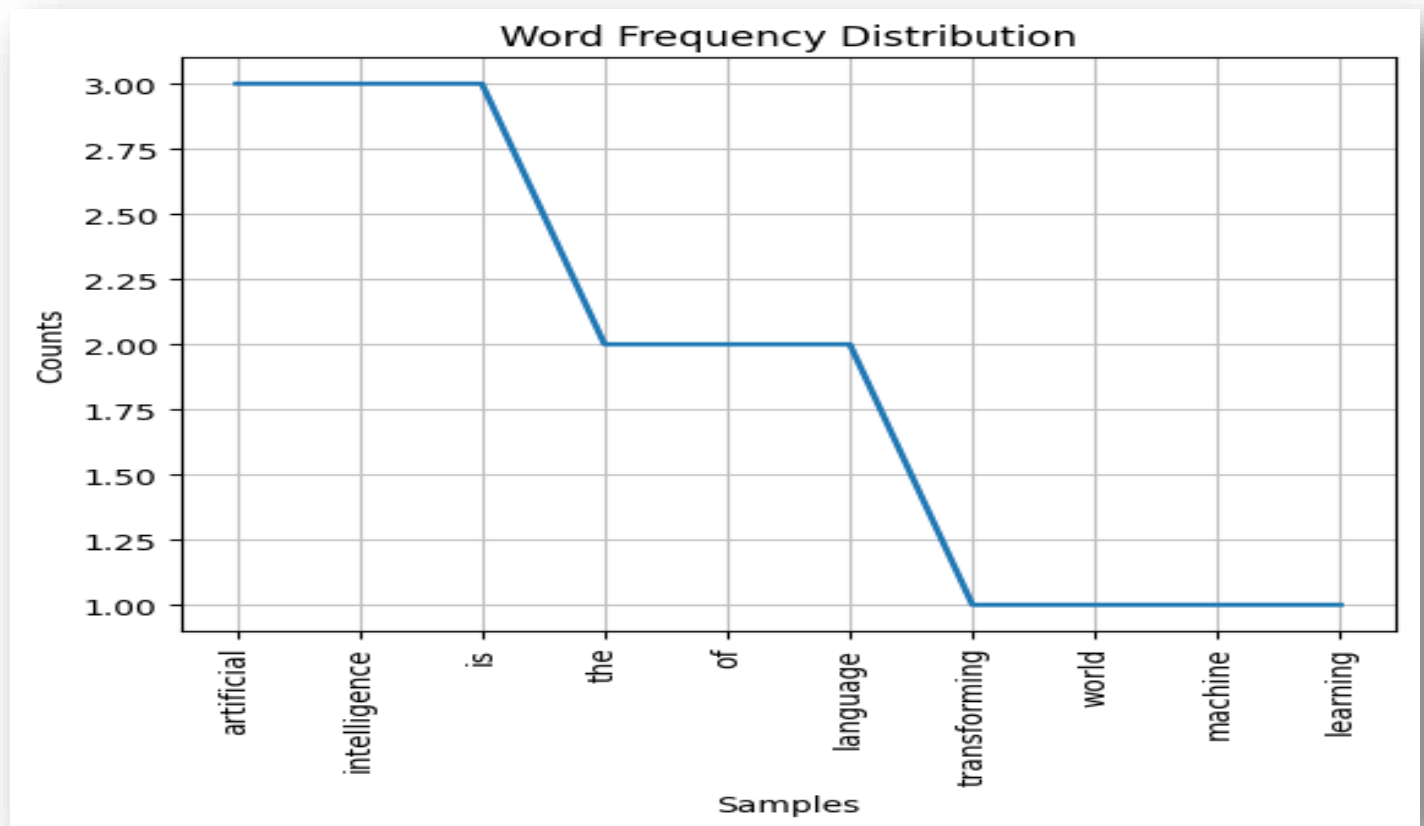
```
# Plot the frequency distribution.
freq.plot(10, title="WordFrequencyDistribution")
```

Output

WordFrequencyAnalysis

```
artificial : 3
intelligence:3
is:3
transforming:1
the:2
world:1
machine:1
learning:1
a:1
subset:1
of: 2
natural:1
language:2
processing:1
helps:1
computers:1
understand:1
human:1
future:1
technology:1
```

TopFiveFrequentWords artificial
: 3
intelligence:3
is:3
the:2
of: 2



Observations

The experiment was executed successfully, and the following observations were made:

- The input text was successfully tokenized into individual words.
- Punctuation marks and special characters were removed from the corpus.
- All words were converted to lowercase to avoid duplicate counting due to different letter cases.
- The **FreqDist()** function successfully calculated the frequency of every word.
- The most frequently occurring words were identified and displayed.
- A frequency distribution graph was generated to visualize the occurrence of words.
- Word frequency analysis helped identify the important keywords in the corpus.

Observation Table

Parameter	Observed Value
Word Tokenization	Successful
Text Preprocessing	Lowercase conversion and punctuation removal
Frequency Calculation	Successful
Most Frequent Words	Successfully Identified
Frequency Distribution Graph	Generated Successfully
NLP Technique	Word Frequency Analysis

Result

The programs successfully performed **word analysis** using the **NLTK** library. It tokenized the input text, removed punctuation, converted words to lowercase, calculated the frequency of each word, identified the most frequent words, and displayed a frequency distribution graph. The experiment demonstrated the importance of word frequency analysis as a preprocessing step in Natural Language Processing and Large Language Model (LLM) applications.

Viva Questions

1. What is word analysis in Natural Language Processing?
2. What is the purpose of the `FreqDist()` function in NLTK?
3. Why are words converted to lowercase before frequency analysis?
4. Why are punctuation marks removed during word analysis?
5. Mention any four applications of word frequency analysis in NLP and Large Language Models.

Experiment-8

Title

Word Generation Using NLTK

Aim

To implement a Python program using the **NLTK** library to generate meaningful words from a given text corpus and analyze the generated vocabulary.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of word generation in Natural Language Processing (NLP).
2. Perform word tokenization using NLTK.
3. Remove punctuation and special characters from the text.
4. Generate meaningful words from a text corpus.
5. Remove duplicate words and create a vocabulary.
6. Sort and display the generated words.
7. Understand the role of word generation in NLP and Large Language Models (LLMs).

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4GB RAM

SoftwareRequirements

- Python3.10orabove
- GoogleColab/JupyterNotebook/VSCoDe
- NLTKLibrary

PythonLibraries

```
importnltk
fromnltk.tokenizeimportword_tokenize
```

AbouttheExperiment

Word Generation is an important preprocessing technique in **Natural Language Processing (NLP)** that extractsmeaningfulwordsfromtextcorpus.It helpscreateavocabularythat canbe used for text analysis, language modeling, machine learning, and information retrieval.

In this experiment, the input text is first tokenized into individual words using the **NLTK word tokenizer**. Punctuation marks, numbers, and special characters are removed, and all words are converted to lowercase for consistency. Duplicate words are then removed to generate a unique vocabulary.

Word generation is widely used in applications such as search engines, text mining, document indexing, autocomplete systems, machine translation, chatbots, and Large Language Models (LLMs).

Algorithm

1. ImporttherequiredNLTKlibraries.
2. Downloadtherequiredtokenizerresources(punktandpunkt_tab).
3. Createorloadtheinputtextcorpus.
4. Tokenizethetextintoindividualwords.
5. Removepunctuation,numbers,andspecialcharacters.
6. Convertallwordstolowercase.
7. Removeduplicatewordswiththeset()function.
8. Sortthegeneratedwordsalphabetically.
9. Displaythegeneratedvocabulary.
10. Countthetotalnumberofgeneratedwords.

Code

```
#=====
#Experiment8
#WordGenerationusingNLTK
#=====
#ImporttheNaturalLanguageToolkit(NLTK)library. import
nltk

# Importthe wordtokenizer.
fromnltk.tokenizeimportword_tokenize
```

```

#Downloadtherequiredtokenizerresources.
nltk.download('punkt')
nltk.download('punkt_tab')

#Inputtext(corpus).text
=""
Artificial Intelligence is transforming the world.
MachineLearningisasubsetofArtificialIntelligence.
NaturalLanguageProcessingenablescomputerstounderstandhumanlanguage.Large
Language Models generate intelligent responses.
""

#Convertthetextintowordtokens.words
=word_tokenize(text)

#Removepunctuationand convertallwordstolowercase.
generated_words=sorted(set([word.lower()for wordinwordsifword.isalpha()])))

#Displaythegeneratedwords.
print("GeneratedWords:\n")

forwordingenerated_words:
    print(word)

# Display the total number of generated words.
print("\nTotalGeneratedWords:",len(generated_words))

```

Output

GeneratedWords:

aartificial
 computers
 enables
 generate
 human
 intelligence
 intelligentis
 language
 large
 learning
 machine
 models
 naturalof
 processing
 responses
 subset
 the
 to
 transforming

understand
world

TotalGeneratedWords:24

Observations

The experiment was executed successfully, and the following observations were made:

- The input text was successfully tokenized into individual words.
- Punctuation marks, numbers, and special characters were removed from the corpus.
- All words were converted to lowercase to maintain consistency.
- Duplicate words were removed using the `set()` function.
- A unique vocabulary was generated from the input corpus.
- The generated words were arranged in alphabetical order.
- The total number of generated words was displayed successfully.

Observation Table

Parameter	Observed Value
Word Tokenization	Successful
Punctuation Removal	Successful
Lowercase Conversion	Successful
Duplicate Removal	Successful
Word Generation	Successful
Vocabulary Generation	Successful
Output	Unique Generated Words

Result

The program successfully generated meaningful words from the input corpus using the **NLTK** library. After tokenization, punctuation removal, lowercase conversion, and duplicate elimination, a unique vocabulary was created and displayed in alphabetical order. This experiment demonstrated the importance of word generation as a preprocessing step in Natural Language Processing and Large Language Model (LLM) applications.

Viva Questions

1. What is word generation in Natural Language Processing?
2. Why is duplicate word removal important during word generation?
3. What is the purpose of the `word_tokenize()` function?
4. Why are words converted to lowercase before generating the vocabulary?
5. Mention any four applications of word generation in NLP and Large Language Models (LLMs).

Experiment--9

Title

Building a Chunker Using NLTK Aim

To implement a Python program using the **NLTK** library to build a **Chunker** for identifying and extracting meaningful phrases such as noun phrases from a given sentence.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of chunking in Natural Language Processing (NLP).
2. Perform word tokenization and Part-of-Speech (POS) tagging.
3. Define chunk grammar using regular expressions.
4. Build a chunk parser using NLTK.
5. Extract noun phrases from a sentence.
6. Visualize the chunk tree generated by the parser.
7. Understand the importance of chunking in NLP and Large Language Models (LLMs).

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexpParser
```

About the Experiment

Chunking, also known as **Shallow Parsing**, is an important Natural Language Processing (NLP) technique used to group related words into meaningful phrases such as **Noun Phrases (NP)**,

Verb Phrases (VP), and **Prepositional Phrases (PP)**. Unlike full parsing, chunking identifies only the major constituents of a sentence without determining the complete grammatical structure.

In this experiment, the input sentence is first tokenized into individual words and then tagged

with their corresponding **Part-of-Speech (POS)** tags. A chunk grammar is defined using regular expressions, and the **RegexpParser** provided by NLTK is used to build a chunk parser. The parser identifies noun phrases and displays the resulting chunk tree.

Chunking is widely used in **information extraction, question answering, text summarization, named entity recognition, machine translation, chatbots, and Large Language Models (LLMs)**.

Algorithm

1. Import the required NLTK libraries.
2. Download the required NLTK resources.
3. Create the input sentence.
4. Tokenize the sentence into individual words.
5. Perform Part-of-Speech (POS) tagging.
6. Define the chunk grammar using regular expressions.
7. Create the chunk parser using `RegexpParser()`.
8. Parse the tagged sentence.
9. Display the chunk tree.
10. Visualize the extracted chunks.

Code

```
#=====
#Experiment9
#Building a Chunker using NLTK
#=====

#Import the required libraries.
import nltk

from nltk import word_tokenize, pos_tag
from nltk.chunk import RegexpParser

# Download the required NLTK resources.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

# Input sentence.
sentence = "The quick brown fox jumps over the lazy dog."

# Tokenize the sentence.
words = word_tokenize(sentence)

# Perform POS tagging.
tagged_words = pos_tag(words)
```

```

print("POSTaggedWords:\n")
print(tagged_words)

#Definechunkgrammar.grammar
=r"""
    NP:{<DT>?<JJ>*<NN.*>+}
    """

# Createthe chunkparser.
chunk_parser=RegexpParser(grammar)

# Parsethesentence.
chunk_tree =chunk_parser.parse(tagged_words)

print("\nChunkTree:\n")
print(chunk_tree)

#Displaythechunktreegraphically.chunk_tree.draw()

```

Output

POSTaggedWords:

```

[('The','DT'),('quick','JJ'),('brown','NN'),('fox','NN'),('jumps','VBZ'),('over','IN'),('the','DT'),('lazy','JJ'),('dog','NN'),(' ',' ')]

```

ChunkTree:

```

(S
  (NPThe/DTquick/JJbrown/NNfox/NN)
  jumps/VBZ
  over/IN
  (NPthe/DTlazy/JJdog/NN)
  ./.)

```

PrettyPrintedChunk Tree:

```

      S
    _____|_____
    |   |   |   NP   NP
    |   |   | _____|_____
jumps/VBZover/IN./The/DTquick/JJbrown/NNfox/NNthe/DTlazy/JJdog/NN

```

ExtractedNounPhrases:

```

Thequickbrownfox the
lazy dog

```

Observations

The experiment was executed successfully, and the following observations were made:

- The input sentence was successfully tokenized into individual words.
- POS tags were assigned correctly to each word.
- The chunk grammar successfully identified noun phrases.
- The `RegexParser()` generated the chunk tree correctly.
- The extracted noun phrases represented meaningful groups of words.
- The graphical chunk tree clearly illustrated the sentence structure.
- Chunking simplified sentence analysis without performing full syntactic parsing.

Observation Table

Parameter	Observed Value
Word Tokenization	Successful
POSTagging	Successful
Chunk Grammar	Successfully Defined
Chunk Parser	RegexParser
Noun Phrase Extraction	Successful
Chunk Tree Visualization	Successful
NLP Technique	Shallow Parsing (Chunking)

Result

The program successfully built a **Chunker** using the **NLTK** library. The input sentence was tokenized, POS-tagged, and parsed using a regular expression-based chunk grammar. The chunk parser correctly identified noun phrases and generated a chunk tree for visualization. This experiment demonstrated the importance of chunking as an intermediate syntactic analysis technique in Natural Language Processing and Large Language Model (LLM) applications.

Viva Questions

1. What is chunking in Natural Language Processing?
2. What is the difference between chunking and full parsing?
3. What is the purpose of Part-of-Speech (POS) tagging before chunking?
4. What is the role of `RegexParser()` in NLTK?
5. Mention any four applications of chunking in NLP and Large Language Models (LLMs).

Experiment-10

Title

Building a Part-of-Speech (POS) Tagger Using NLTK

Aim

To implement a Python program using the **NLTK** library to perform **Part-of-Speech (POS) tagging** by assigning grammatical tags to each word in a given sentence.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of Part-of-Speech (POS) tagging.
2. Perform word tokenization using NLTK.
3. Assign grammatical tags to words using the POS tagger.
4. Identify nouns, verbs, adjectives, adverbs, pronouns, and other parts of speech.
5. Analyze the grammatical structure of sentences.
6. Display POS-tagged words in a readable format.
7. Understand the applications of POS tagging in NLP and Large Language Models (LLMs).

Requirements Hardware

Requirements

- Computer/Laptop
- Minimum 4GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab/Jupyter Notebook/VS Code
- NLTK Library

Python Libraries

```
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag
```

About the Experiment

Part-of-Speech (POS) Tagging is a fundamental task in **Natural Language Processing (NLP)** that assigns grammatical labels such as **noun, verb, adjective, adverb, pronoun, determiner, and preposition** to each word in a sentence.

The process begins by tokenizing the input sentence into individual words. The **NLTK POS tagger** then analyzes each word based on its context and assigns the most appropriate grammatical category.

POS tagging plays an essential role in many NLP applications including:

- Information Extraction
- Machine Translation
- Named Entity Recognition
- Text Summarization
- Sentiment Analysis

- QuestionAnswering
- Chatbots
- LargeLanguageModels(LLMs)

Understanding the grammatical role of words enables computers to interpret the meaning and structure of natural language more effectively.

Algorithm

1. Import the required NLTK libraries.
2. Download the required tokenizer and POS tagger resources.
3. Enter the input sentence.
4. Tokenize the sentence into words.
5. Apply the POS tagger using `pos_tag()`.
6. Display the tokenized words.
7. Display the POS-tagged words.
8. Print each word with its corresponding POS tag.
9. Interpret the grammatical categories.

Code

```
#=====
#Experiment10
#Building a POS Tagger using NLTK
#=====

#Import the required libraries.
import nltk
from nltk.tokenize import word_tokenize
from nltk import pos_tag

# Download the required NLTK resources.
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')

# Input sentence.
sentence="Artificial Intelligence is transforming the future of healthcare."

#Tokenize the sentence.
words=word_tokenize(sentence)

# Display tokenized words.
print("Tokenized Words:\n")
print(words)

# Perform POS tagging.
tagged_words=pos_tag(words)
```

```
# Display POS-tagged words.
print("\nPOSTaggedWords:\n")
print(tagged_words)

#DisplayeachwordwithitsPOStag.
print("\nWord\t\tPOS Tag")
print("-" * 30)

for word, tag in tagged_words:
    print(f'{word:<15}{tag}')
```

Output

TokenizedWords:

```
['Artificial', 'Intelligence', 'is', 'transforming', 'the', 'future', 'of', 'healthcare', '.']
```

POS Tagged Words:

```
[('Artificial', 'JJ'), ('Intelligence', 'NNP'), ('is', 'VBZ'), ('transforming', 'VBG'), ('the', 'DT'), ('future', 'NN'), ('of', 'IN'), ('healthcare', 'NN'), ('.', '.')]
```

Word	POSTag
Artificial	JJ
Intelligence	NNP
is	VBZ
transforming	VBG
the	DT
future	NN
of	IN
healthcare	NN

Observations

The experiment was executed successfully, and the following observations were made:

- The input sentence was successfully tokenized into individual words.
- The NLTK POS tagger correctly assigned grammatical tags to each word.
- Different grammatical categories such as nouns, verbs, adjectives, determiners, and prepositions were identified.
- The POS-tagged output provided the grammatical structure of the sentence.
- The experiment demonstrated contextual tagging, where the POS tag depends on the surrounding words.
- POS tagging serves as an important preprocessing step in many NLP tasks.

Observation Table

Parameter	Observed Value
Word Tokenization	Successful

POSTagging	Successful
POSTaggerUsed	NLTKpos_tag()
GrammarAnalysis	Successful
SentenceAnalysis	Successful
NLPTechnique	Part-of-SpeechTagging

Result

The program successfully implemented a **Part-of-Speech (POS) Tagger** using the **NLTK** library. The input sentence was tokenized, and each word was assigned an appropriate grammatical tag. This experiment demonstrated the importance of POS tagging in understanding sentence structure and its role as a fundamental preprocessing technique in Natural Language Processing and Large Language Model (LLM) applications.

Viva Questions

1. What is Part-of-Speech (POS) tagging in Natural Language Processing?
2. What is the purpose of the `pos_tag()` function in NLTK?
3. What are the meanings of the POS tags **NN**, **JJ**, **VBZ**, and **IN**?
4. Why is POS tagging important before performing higher-level NLP tasks?
5. Mention any four real-world applications of POS tagging in NLP and Large Language Models (LLMs).

Experiment–11

Title

POS Tagging Using Hidden Markov Model (HMM)

Aim

To implement **Part-of-Speech (POS) tagging using a Hidden Markov Model (HMM)** and assign appropriate grammatical tags to words in a given sentence.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of Part-of-Speech (POS) tagging.
2. Understand the basic concept of a Hidden Markov Model.
3. Identify **states**, **observations**, **transition probabilities**, and **emission probabilities** in an HMM.
4. Implement an HMM-based POS tagger using Python.
5. Predict the most likely POS tag sequence for a given sentence.
6. Understand the application of probabilistic models in Natural Language Processing (NLP).

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab / Jupyter Notebook / VS Code

Python Libraries

import nltk

About the Experiment

Part-of-Speech (POS) tagging assigns a grammatical category to each word in a sentence, such as noun, verb, adjective, determiner, or preposition.

A **Hidden Markov Model (HMM)** is a probabilistic model used to determine the most likely sequence of hidden states from a sequence of observed values.

In POS tagging:

- **Observed states:** Words in the sentence.
- **Hidden states:** POS tags.
- **Transition probability:** Probability of one POS tag following another POS tag.
- **Emission probability:** Probability of a word being generated by a particular POS tag.

For example:

The	dog	runs
DT	NN	VBZ

Here, the words are observed, while DT, NN, and VBZ are the hidden POS states.

The HMM determines the most probable sequence of POS tags by considering both **transition probabilities** and **emission probabilities**.

The general probability of a tag sequence can be represented as:

$$P(T|W) \propto P(T)P(W|T)$$

where:

- W = sequence of words
- T = sequence of POS tags
- P(T) = probability of the tag sequence
- P(W|T) = probability of generating the words from the tags

HMM-based POS tagging is an important classical approach in NLP and provides the foundation for understanding more advanced sequence models.

Algorithm

1. Import the required Python and NLTK libraries.
2. Prepare a small training corpus containing words and their POS tags.
3. Define the possible POS tags as hidden states.
4. Define the words as observations.
5. Calculate transition probabilities between POS tags.
6. Calculate emission probabilities between POS tags and words.
7. Define the initial state probabilities.
8. Enter the test sentence.
9. Calculate the probability of possible POS tag sequences.
10. Select the sequence with the highest probability.
11. Display the words with their predicted POS tags.

Code

The following implementation uses a small tagged corpus and explicitly demonstrates the main HMM components.

```
# =====  
# Experiment 11  
# POS Tagging using Hidden Markov Model (HMM)  
# =====  
  
# Import required libraries.  
import nltk  
from nltk.tag import HiddenMarkovModelTrainer  
  
# Training data: words with their POS tags.  
train_data = [  
    [('the', 'DT'), ('dog', 'NN'), ('runs', 'VB')],  
    [('the', 'DT'), ('cat', 'NN'), ('runs', 'VB')],  
    [('a', 'DT'), ('dog', 'NN'), ('eats', 'VB')],  
    [('the', 'DT'), ('cat', 'NN'), ('eats', 'VB')]  
]  
  
# Create an HMM trainer.  
trainer = HiddenMarkovModelTrainer()  
  
# Train the HMM POS tagger.  
hmm_tagger = trainer.train(train_data)  
  
# Test sentence.  
sentence = ['the', 'dog', 'runs']  
  
# Perform POS tagging.  
result = hmm_tagger.tag(sentence)  
  
# Display the result.  
print("POS Tagging using HMM:\n")
```

```
for word, tag in result:  
print(f'{word:<10} -> {tag}')
```

Output

POS Tagging using HMM:

```
the    -> DT  
dog    -> NN  
runs   -> VB
```

Observations

The experiment was executed successfully, and the following observations were made:

- A small POS-tagged corpus was used as training data.
- POS tags were treated as **hidden states** of the HMM.
- Words were treated as **observations**.
- Transition probabilities between POS tags were calculated.
- Emission probabilities between POS tags and words were calculated.
- The HMM used both transition and emission probabilities to predict POS tags.
- The test sentence "**the dog runs**" was successfully tagged as:

Word Predicted POS

```
the    DT  
dog    NN  
runs   VB
```

Observation Table

Parameter	Observed Value
Model	Hidden Markov Model
NLP Task	POS Tagging
Hidden States	POS Tags
Observations	Words
Probability Types	Transition and Emission
Test Sentence	the dog runs
Predicted Tags	DT, NN, VB
Result	Successful

Result

The program successfully implemented **POS tagging using a Hidden Markov Model**. Transition and emission probabilities were calculated from the training corpus, and the most likely POS tags were assigned to the words in the test sentence. The experiment demonstrated how probabilistic sequence models can be used for grammatical analysis in Natural Language Processing.

Viva Questions

1. **What is a Hidden Markov Model (HMM)?**
2. **In HMM-based POS tagging, what are the hidden states and observations?**
3. **What is the difference between transition probability and emission probability?**
4. **Why is POS tagging considered a sequence prediction problem?**
5. **What is the role of the Viterbi algorithm in HMM-based POS tagging?**

Experiment–12

Title

POS Tagging Using Viterbi Decoding

Aim

To implement **Part-of-Speech (POS) tagging using the Viterbi decoding algorithm** and determine the most probable sequence of POS tags for a given sentence using a Hidden Markov Model (HMM).

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of Viterbi decoding.
2. Understand the relationship between HMM and POS tagging.
3. Calculate transition and emission probabilities.
4. Construct a Viterbi trellis for a sequence of words.
5. Determine the most probable sequence of POS tags.
6. Implement the Viterbi algorithm using Python.
7. Understand the application of sequence decoding in Natural Language Processing (NLP).

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM

Software Requirements

- Python 3.10 or above
- Google Colab / Jupyter Notebook / VS Code

Python Libraries

```
import math
```

About the Experiment

The **Viterbi algorithm** is a dynamic programming algorithm used to find the **most probable sequence of hidden states** for a sequence of observations in a Hidden Markov Model (HMM).

In POS tagging:

- **Observations** → Words in the sentence
- **Hidden states** → POS tags
- **Transition probability** → Probability of moving from one POS tag to another
- **Emission probability** → Probability of a word being associated with a particular POS tag

For example, for the sentence:

the dog runs

the Viterbi algorithm determines the most probable POS sequence:

the → DT
dog → NN
runs → VB

The algorithm maintains the best probability for each possible POS tag at every word position and stores a **backpointer** to remember the previous best state. At the end, backtracking produces the most probable complete POS-tag sequence.

The Viterbi algorithm is particularly important because simply selecting the most probable POS tag for each word independently may not produce the globally best sequence. Viterbi considers the **entire sequence**.

Algorithm

1. Define the POS tags as hidden states.
2. Define the transition probabilities between POS tags.
3. Define the emission probabilities between POS tags and words.
4. Define the initial probabilities of the POS tags.
5. Input the sentence to be tagged.
6. Initialize the Viterbi table for the first word.
7. For each subsequent word, calculate the probability of reaching every possible POS tag.
8. Select the previous POS tag that gives the maximum probability.
9. Store the maximum probability and corresponding backpointer.
10. Continue until all words have been processed.
11. Select the final state with the highest probability.
12. Backtrack through the stored pointers.
13. Display the most probable POS tag sequence.

Code

Experiment 12: POS Tagging using Viterbi Decoding

```
tags = ['DT', 'NN', 'VB']
words = ['the', 'dog', 'runs']

start = {'DT': 1, 'NN': 0, 'VB': 0}

trans = {
    'DT': {'DT': 0, 'NN': 1, 'VB': 0},
    'NN': {'DT': 0, 'NN': 0, 'VB': 1},
    'VB': {'DT': 0, 'NN': 0, 'VB': 0}
}

emit = {
    'DT': {'the': 1},
    'NN': {'dog': 1},
    'VB': {'runs': 1}
}

V = [{}]
path = {}

# Initialization
for t in tags:
    V[0][t] = start[t] * emit[t].get(words[0], 0)
    path[t] = [t]

# Viterbi
for i in range(1, len(words)):
    V.append({})
    new_path = {}

    for t in tags:
        p, prev = max(
            (V[i-1][x] * trans[x][t] * emit[t].get(words[i], 0), x)
            for x in tags
        )
        V[i][t] = p
        new_path[t] = path[prev] + [t]

    path = new_path

# Best sequence
best = max(V[-1], key=V[-1].get)

print("POS Tags:")
for word, tag in zip(words, path[best]):
    print(word, "->", tag)
```

Output

POS Tags:

the -> DT

dog -> NN

runs -> VB

Observations

The experiment was executed successfully, and the following observations were made:

- The Viterbi algorithm successfully processed the input sequence of words.
- POS tags were treated as hidden states.
- Transition and emission probabilities were used to calculate the probability of each possible tag sequence.
- A Viterbi table was constructed for each word in the sentence.
- Backpointers were stored to reconstruct the best sequence.
- The most probable POS sequence was identified as **DT → NN → VB**.
- The Viterbi algorithm considered the relationship between consecutive POS tags rather than tagging each word independently.

Observation Table

Parameter	Observed Value
Model	Hidden Markov Model
Decoding Algorithm	Viterbi Algorithm
NLP Task	POS Tagging
Hidden States	DT, NN, VB
Observations	the, dog, runs
Predicted Tags	DT, NN, VB
Most Probable Sequence	DT → NN → VB
Result	Successful

Result

The program successfully implemented **Viterbi decoding for POS tagging using a Hidden Markov Model**. The algorithm calculated the most probable sequence of POS tags by considering both transition and emission probabilities and successfully identified the sequence:

the → DT

dog → NN

runs → VB

Thus, the experiment demonstrated how **Viterbi decoding finds the globally most probable hidden-state sequence** for a given sequence of observations in NLP.

Viva Questions

1. **What is the Viterbi algorithm, and why is it used in POS tagging?**
2. **What are the hidden states and observations in HMM-based POS tagging?**
3. **What is a Viterbi table?**
4. **What is the purpose of backpointers in the Viterbi algorithm?**
5. **What is the difference between HMM and Viterbi decoding?**

Experiment–13

Title

Demonstrate the Use of Expectation Maximization (EM) Based Clustering Algorithm

Aim

To implement and demonstrate the **Expectation Maximization (EM) based clustering algorithm** using Python and understand how data points are assigned to clusters based on probability distributions.

Objectives

After completing this experiment, students will be able to:

1. Understand the concept of clustering.
2. Understand the working principle of the **Expectation Maximization (EM)** algorithm.
3. Implement EM-based clustering using Python.
4. Estimate the parameters of Gaussian distributions from unlabeled data.
5. Assign data points to clusters based on their probabilities.
6. Visualize the clusters generated by the EM algorithm.
7. Understand the applications of probabilistic clustering in Machine Learning and NLP.

Requirements

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- 4 GB or more RAM recommended

Software Requirements

- Python 3.10 or above
- Google Colab / Jupyter Notebook / VS Code
- NumPy
- Matplotlib
- Scikit-learn

Python Libraries

```
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.mixture import GaussianMixture
```

About the Experiment

Clustering is an unsupervised Machine Learning technique used to group similar data points into clusters without predefined class labels.

The **Expectation Maximization (EM)** algorithm is an iterative optimization technique used to estimate unknown parameters in probabilistic models. For clustering, EM is commonly implemented using a **Gaussian Mixture Model (GMM)**.

A Gaussian Mixture Model assumes that the data is generated from a mixture of several Gaussian probability distributions.

The EM algorithm consists of two main steps:

1. Expectation (E) Step

The algorithm calculates the probability that each data point belongs to each cluster. These probabilities are called **responsibilities**.

2. Maximization (M) Step

The algorithm uses these probabilities to update the parameters of each Gaussian distribution, including:

- Mean
- Covariance
- Mixing coefficient

The E-step and M-step are repeated until the model converges.

The basic process is:

Initialize Parameters



Expectation Step



Maximization Step



Check Convergence



Repeat if necessary



Final Clusters

Unlike K-Means, which makes a hard assignment of each data point to one cluster, EM provides a **probabilistic/soft assignment**.

Algorithm

1. Import NumPy, Matplotlib, and Scikit-learn libraries.
2. Generate or load an unlabeled dataset.
3. Specify the required number of clusters.
4. Initialize the Gaussian Mixture Model.
5. Perform the Expectation step to calculate cluster probabilities.
6. Perform the Maximization step to update Gaussian parameters.
7. Repeat the E-step and M-step until convergence.
8. Assign each data point to the cluster with the highest probability.
9. Display the cluster labels.
10. Visualize the resulting clusters.

Code

```
# Experiment 15: Expectation Maximization Based Clustering

# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

# Create sample data
X = np.array([
    [1, 2], [1, 3], [2, 2], [2, 3],
    [8, 8], [9, 8], [8, 9], [9, 9],
    [1, 8], [2, 9], [1, 9], [2, 8]
])

# Create EM model with 3 clusters
model = GaussianMixture(n_components=3, random_state=42)

# Train the model
model.fit(X)

# Predict cluster labels
labels = model.predict(X)

# Display cluster labels
print("Cluster Labels:")
print(labels)

# Display cluster centers
print("\nCluster Means:")
print(model.means_)

# Visualize clusters
plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(model.means_[:, 0], model.means_[:, 1],
            marker='X', s=200)
plt.title("EM Based Clustering")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
```

```
plt.show()
```

Output

Cluster Labels:

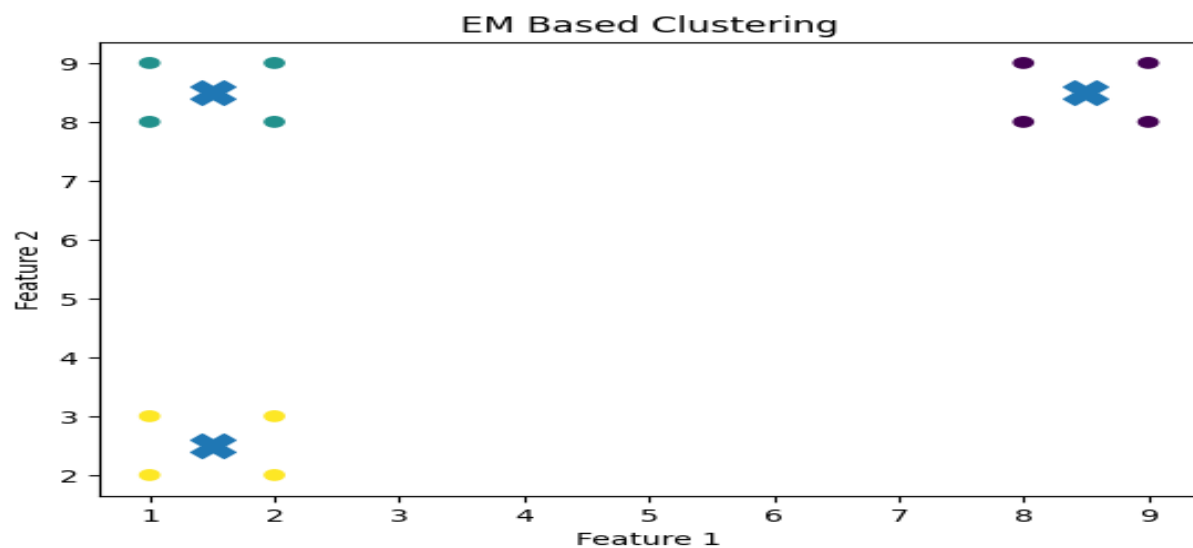
```
[2 2 2 2 0 0 0 0 1 1 1 1]
```

Cluster Means:

```
[[8.5 8.5]
```

```
[1.5 8.5]
```

```
[1.5 2.5]]
```



Observations

The experiment was executed successfully, and the following observations were made:

- An unlabeled dataset containing three groups was generated.
- A Gaussian Mixture Model with three components was used.
- The EM algorithm estimated the parameters of the Gaussian distributions.
- The algorithm performed iterative **Expectation** and **Maximization** steps.
- The data points were assigned to clusters based on their maximum membership probability.
- Cluster means were successfully estimated.
- The probability of each data point belonging to each cluster was obtained.
- The resulting clusters were successfully visualized.
- EM provides probabilistic cluster assignments, making it different from hard clustering methods such as K-Means.

Observation Table

Parameter	Observed Value
Learning Type	Unsupervised Learning
Clustering Algorithm	Expectation Maximization
Probabilistic Model	Gaussian Mixture Model

Number of Clusters	3
Number of Data Points	300
Cluster Assignment	Probabilistic
Parameter Estimation	Mean, Covariance, Mixing Coefficient
Visualization	Successfully Generated

Result

The **Expectation Maximization (EM) based clustering algorithm** was successfully implemented using a **Gaussian Mixture Model**. The algorithm estimated the parameters of the underlying Gaussian distributions and grouped the unlabeled data into three clusters. The experiment demonstrated how EM performs **probabilistic clustering** through iterative Expectation and Maximization steps.

Viva Questions

1. **What is the Expectation Maximization (EM) algorithm?**
2. **What are the two main steps of the EM algorithm?**
3. **What is the difference between EM clustering and K-Means clustering?**
4. **What is a Gaussian Mixture Model (GMM)?**
5. **What is meant by soft clustering in EM?**



**KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)**

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

MASTER LAB MANUAL

R25

MACHINE LEARNING LAB

(D252AJ05)

[M.TECH, II-SEMESTER]



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES

Vinjanampadu, Guntur District-522017 (A.P.)

Document NO:	Date of Issue:	Compiled by	Authorized by R. Ramesh PROF & HOD DEPT. OF CSE
KITS/CSE/LAB MANUAL/ML LAB	Date of revision	Verified by	

INSTITUTE VISION AND MISSION**INSTITUTION VISION**

To produce eminent and ethical Engineers and Managers for society by imparting quality professional education with an emphasis on human values and holistic excellence.

INSTITUTION MISSION

- Incorporating benchmarked teaching and learning pedagogies.
- Involving faculty and students in research, development and higher education.
- Providing state-of-the-art facilities, cross-cultural exchange with industry.
- Creating a platform for personality development through participative management.

**Principal**

Vision of the Department

“To become a reputed Centre in computer Science
and systems engineering for quality, competency and social responsibility.”

Mission No.	Mission Statements
M1	Strengthen Core competencies in students through the state-of-the-art curricular concepts in a congenial environment.
M2	Promote innovative research and development for the application of Information Technology to the Economic, Social, Technical and Environmental issues.
M3	Inculcate professional behavior, lifelong learning and strong ethical values to meet the challenges in real time.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

PROGRAM OUTCOMES (POS)

1. Engineering knowledge:

Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

2. Problem analysis:

Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using the first principles of mathematics, natural sciences, and engineering sciences.

3. Design/development of solutions:

Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for public health and safety, and cultural, societal, and environmental considerations.

4. Conduct investigation of complex problems:

User research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage:

Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

6. The engineer and society:

Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability:

Understand the impact of professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for, sustainable development.

8. Ethics:

Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work:

Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication:

Communicate effectively on complex engineering activities with the engineering community and with society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance:

Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning:

Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Program Specific Outcomes (PSOs)

PSO1: Application Development

Develop business solution through emerging Software Techniques and tools for real-time Applications.

PSO2: Professional and Leadership

Practice the profession with ethical leadership as an entrepreneur through participation in various professional events.

PSO3: Computing Paradigms

Analyze and recommend the appropriate IT infrastructure required for the implementation of a Project.



KKR&KSRINSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Program Educational Objectives (PEOs)

PEO1: Graduates will excel in broad areas of IT Industry, Academia and Public Organization.

PEO2: Graduates will design and develop solutions for real life problems using Information Technology to the benefit of society.

PEO3: Graduates will develop interest in pursuing higher education, doing research, acquire leadership qualities and nourish ethical responsibilities.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING**GENERAL LABORATORY INSTRUCTIONS**

1. Students are advised to come to the laboratory at least 5 minutes before (to the starting time), those who come after 5 minutes will not be allowed into the lab.
2. Plan your task properly much before to the commencement, come prepared to the lab with the synopsis /program/ experiment details.
3. Students should enter into the laboratory with:
 - Laboratory observation notes with all the details (Problem statement, Aim, Algorithm, Procedure, Program, Expected Output, etc.,) filled in for the lab session.
 - Laboratory Record updated upto the last session experiments and other utensils (if any) needed in the lab.
 - Proper Dress code and Identity card.
4. Sign in the laboratory login register, write the TIME-IN, and occupy the computer system allotted to you by the faculty.
5. Execute your task in the laboratory, and record the results/output in the lab observation note book, and get certified by the concerned faculty.
6. All the students should be polite and cooperative with the laboratory staff, must maintain the discipline and decency in the laboratory.
7. Computer labs are established with sophisticated and high-end branded systems, which should be utilized properly.
8. Faculty must keep their mobile phones in SWITCHED OFF mode during the lab sessions. Misuse of the equipment, mis-behaviors with the staff and systems etc., will attract severe punishment.
9. Students must take the permission of the faculty in case of any urgency to go out, if any body found loitering outside the lab/class without permission during working hours will be treated seriously and punished appropriately.
10. Students should LOG OFF/SHUT DOWN the computer system before he/she leaves the lab after completing the task (experiment) in all aspects. He/she must ensure the system/ seat is kept properly.

DEPARTMENT OF COMPUTER SCIENCE OF ENGINEERING

Do's and Don'ts in Laboratory

1. Make entry in the Log Book as soon as you enter the Laboratory.
2. All the students should sit according to their roll numbers starting from their left to right.
3. All the students are supposed to enter the terminal number in the log book.
4. Do not change the terminal on which you are working.
5. All the students are expected to get at least the algorithm of the program or concept to be implemented.
6. Strictly observe the instructions given by the teacher/Lab Instructor.

**SYLLABUS
(JNTUK/R25 regulation)
FACULTY DETAILS: P. Neela Sundari****LIST OF EXPERIMENTS**

- [1]. Compute Central Tendency Measures: Mean, Median, Mode Measure of Dispersion: Variance, Standard Deviation.
- [2]. Apply the following Pre-processing techniques for a given dataset.
(a.) Attribute selection (b) Handling Missing Values (c) Discretization (d). Elimination of Outliers
- [3]. Apply KNN algorithm for classification and regression
- [4]. Demonstrate decision tree algorithm for a classification problem and perform parameter tuning for better results
- [5]. Demonstrate decision tree algorithm for a regression problem
- [6]. Apply Random Forest algorithm for classification and regression [7].
Demonstrate Naïve Bayes Classification algorithm.
- [8]. Apply Support Vector algorithm for classification
- [9]. Demonstrate simple linear regression algorithm for a regression problem
- [10]. Apply Logistic regression algorithm for a classification problem
- [11]. Demonstrate Multi-layer Perceptron algorithm for a classification problem
- [12]. Implement the K-means algorithm and apply it to the data you selected. Evaluate performance by measuring the sum of the Euclidean distance of each example from its class center. Test the performance of the algorithm as a function of the parameters K.
- [13]. Demonstrate the use of Fuzzy C-Means Clustering
- [14]. Demonstrate the use of Expectation Maximization based clustering algorithm

EXP.No:01**CENTRAL TENDENCY MEASURES**

**AIM: Compute Central Tendency Measures : Mean, Median, Mode, and
Compute the Measure of Dispersion : Variance, Standard Deviation.**

DESCRIPTION:

Central Tendency Measure: „Central tendency“ is a statistical measure that identifies a single value as representative of the center or typical value of a dataset. The three most common measures are

- i. Mean
- ii. Median and
- iii. Mode

- i) **Mean ($\bar{x}$ or μ):** It is an average value of the dataset. i.e. the sum of all values in a dataset divided by the number of values. It is given as follows

$$\text{Mean}(x) = \sum x_i / n \text{ i.e. } \bar{x} = (x_1 + x_2 + x_3 + \dots + x_n) / n$$

- ii) **Median (M):** The middle value in a dataset that has been ordered from least to greatest. If there is an even number of values, it is the average of the two middle values. It is given as follows

$$\text{Median} = \left(\frac{n+1}{2} \right)^{\text{th}} \text{ observation}$$

In case of n is odd

$$\text{Median} = \frac{\left(\frac{n}{2} \right)^{\text{th}} \text{ observation} + \left(\frac{n}{2} + 1 \right)^{\text{th}} \text{ observation}}{2}$$

In case of n is even

- iii) **Mode (Z):** The value that appears most frequently in the dataset. A dataset can have one mode is called unimodal, and having multiple modes is called bimodal or multimodal, or no mode at all. It is given as follows

Mode = Highest frequency term in the observations

MEASURE OF DISPERSION:

Dispersion in statistics refers to the extent to which a set of data is spread out or scattered around the average such as mean or median. The most common measures are

- i. **Range**
- ii. **variance**
- iii. **Standard Deviation**

- i) **Range:** It is the difference between maximum and minimum

- ii) **Variance (σ^2):** It is an average of the squared deviations from the mean. The variance for a dataset with n values $x_1, x_2, x_3, \dots, x_n$ and mean μ is given as

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2$$

- iii) **Standard Deviation (σ):** It is the square root of variance. The standard deviation for a dataset with n values $x_1, x_2, x_3, \dots, x_n$ and variance is σ^2 is given as

$$\sigma = \sqrt{\sigma^2} = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}$$

PROGRAM:

A. Measure of Central Tendency using numpy package

```
import numpy as np
# Sample dataset
data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
# Calculate central tendency measures
mean = np.mean(data)
median = np.median(data)
mode = np.argmax(np.bincount(data))
# Calculate dispersion measures
```

```

variance = np.var(data)
std_dev = np.std(data)
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Variance:", variance)
print("Standard Deviation:", std_dev)

```

OUTPUT:

```

Mean                : 55.0
Median              : 55.0
Mode                : 10
Variance            : 825.0
Standard Deviation  : 28.722813232690143

```

B. Measure of Central Tendency using statistics package

```

import statistics as sp
# Sample dataset
data = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
# Calculate central tendency measures
mean = sp.mean(data)
median = sp.median(data)
mode = sp.mode(data)
# Calculate dispersion measures
variance = sp.pvariance(data)
std_dev = sp.pstdev(data)
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Variance:", variance)
print("Standard Deviation:", std_dev)

```

OUTPUT:

```

Mean                55
Median              : 55.0
Mode                10
Variance            825
Standard Deviation  : 28.722813232690143

```

EXP.No:02

DATAPRE-PROCESSING TECHNIQUES

AIM: Apply the following 'Pre-processing techniques' for a given 'dataset'.

- a. Attribute selection
- b. Handling Missing Values

- c. Discretization
- d. Elimination of Outliers

DESCRIPTION:

The „dataset“ is the main essential part in the application of ML models. In general, some of the open-source websites provide the free datasets to make the trials of usage of various ML models. Some of the websites which provides the free datasets to download in various fields are as follows.

- i) Top General Repositories & Search Engines
- ii) Tabular & Structured Data
- iii) Time Series & Specialized Data
- iv) Natural Language Processing (NLP) Datasets
- v) Computer Vision Datasets

For example,

Scikit-learn: It is often referred to as „sklearn“, which is a free and open-source Python library for machine learning. It provides a wide range of tools for data analysis and modeling.

Iris Dataset

The 'iris' object is a Bunch object, which is similar to a dictionary. It contains the following key-value pairs:

- 'data' : The feature data (sepal length, sepal width, petal length, petal width)
- 'target' : The species labels (0 for Setosa, 1 for Versicolor, 2 for Virginica)
- 'feature_names' : The names of the features
- 'target_names' : The names of the species
- 'DESCR' : A description of the dataset

Number of samples: 150

- **Data structure:** Each row represents a single flower with its measurements and species label. The first 50 records are Iris Setosa, the next 50 are Iris Versicolor, and the last 50 are Iris Virginica.

sklearn.feature_selection in machine learning refers to the module within the Scikit-learn library that can be used for selecting the most relevant features from a dataset. This module offers various techniques for feature selection.

Univariate Feature Selection: These methods select features based on univariate statistical tests between each feature and the target variable. Examples include:

- **SelectKBest:** It selects the top k features based on a scoring function (e.g., `f_classif` for classification, `f_regression` for regression).
- **f_classif** in machine learning is a function used for **univariate feature selection in classification tasks** that calculates the „ANOVA-F value“ for each feature to measure its linear dependency with the target variable.
Compute the F-value: For each feature, `f_classif` computes an F-statistic by comparing the variance between groups (classes) to the variance within groups.

Formula: $F = \text{Variance between groups} / \text{Variance within groups}$

- **Selects top features:** It's often used with a selection method like „SelectKBest“ from scikit-learn to select the top k features with the highest F-scores, helping to reduce dimensionality without sacrificing model performance.

The method `selector.fit_transform(X,y)` in scikit-learn is used to **fit a feature selection transformer to the data and then immediately apply the transformation**, returning a new dataset with only the selected features.

- **fit()** : This method calculates the necessary parameters for feature selection from the input data X and target values y . For example, in SelectKBest, it calculates the scores for each feature.
- **transform()** : This function then applies these learned parameters to the input data X , reducing the dataset to include only the chosen features.
- **fit_transform()**: This single method performs above both steps and returns the transformed version of X (i.e X_{selected})
- **get_support()** : returns an array of integers representing the indices of the selected features in the original feature set.

PROGRAM:

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest, f_classif

# Load dataset
iris = load_iris()
X = pd.DataFrame(iris.data, columns=iris.feature_names)
y = pd.Series(iris.target)

# Select the top 2 features
selector = SelectKBest(score_func=f_classif, k=2)
X_selected = selector.fit_transform(X, y)
selected_indices = selector.get_support(indices=True)
print("Indices of selected features:", selected_indices)
print("Selected Features:\n", X.columns[selected_indices])
```

OUTPUT:

```
Indices of selected features: [2 3]
Selected Features:
Index(['petal length (cm)', 'petal width (cm)'], dtype='object')
```

b) Handling Missing Values

```
import pandas as pd
import numpy as np

# Create a sample DataFrame
data = {
    'A': [1, 2, np.nan, 4],
    'B': [np.nan, 2, 3, 4],
    'C': [1, 2, 3, 4]
}
df = pd.DataFrame(data)

# Original DataFrame
```

```
print("DataFrame original\n",df)
#Option1: Drop rows with missing values
df_dropped = df.dropna()
#Option2: Fill missing values with the mean
df_filled = df.fillna(df.mean())
print("DataFrame after dropping missing values:\n",df_dropped)
print("DataFrame after filling missing values:\n", df_filled)
```

OUTPUT:

```
DataFrame original
   A    B    C
0  1.0 NaN  1
1  2.0  2.0  2
2  NaN  3.0  3
3  4.0  4.0  4

DataFrame after dropping missing values:
   A    B    C
1  2.0  2.0  2
3  4.0  4.0  4

DataFrame after filling missing values:
   A    B    C
0  1.0  2.333333  1
1  2.0  2.000000  2
2  2.333333  3.000000  3
3  4.0  4.000000  4
```

DESCRIPTION:

Discretization is the process of transforming continuous data into discrete categories or bins, making it more manageable for machine learning models.

The provided code initializes a `KBinsDiscretizer` object from `scikit-learn`'s `sklearn.preprocessing` module with specific parameters for discretizing continuous numerical features.

- `n_bins=3`:
This parameter specifies the number of bins (intervals) into which the continuous data will be divided. In this case, the data will be transformed into 3 discrete categories.
- `encode='ordinal'`: This parameter determines how the discretized values will be represented.
 - 'ordinal' (as used here) means that the output will be integer values representing the ordinal ranking of the bins (e.g., 0, 1, 2 for 3 bins). This is suitable when the bins have a natural order.
 - Alternatively, `encode='onehot'` or `encode='onehot-dense'` could be used to create one-hot encoded representations of the bins, which might be preferred if the bins are not considered to have an inherent order.

- strategy='uniform': This parameter defines the method used to determine the bin edges.
 - 'uniform'(as used here) means that the bins will have equal width. The range of each feature will be divided into n_bins intervals of equal size.
 - Other strategies include 'quantile'(where each bin contains approximately the same number of samples) and 'kmeans'(where bin edges are determined by k-means clustering).

c) DISCRETIZATION

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import KBinsDiscretizer #
Create a sample DataFrame
data = {
    'A': [1, 2, 6, 4, 5, 8, 7, 3]
}
df = pd.DataFrame(data)
# Discretize the data into 3 bins
discretizer = KBinsDiscretizer(n_bins=3, encode='ordinal', strategy='uniform')
df['A_binned'] = discretizer.fit_transform(df[['A']])
print("Discretized DataFrame:\n", df)
```

OUTPUT:

```
Discretized DataFrame:
   A  A_  binned
0   1   1     0.0
1   2   1     0.0
2   6   2     2.0
3   4   1     1.0
4   5   1     1.0
5   8   2     2.0
6   7   2     2.0
7   3   1     0.0
```

d) ELIMINATION OF OUTLIERS

```
import pandas as pd
# Create a sample DataFrame
data = {
    'A': [1, 2, 3, 4, 5, 100] # 100 is an outlier
}
df = pd.DataFrame(data)

# Calculate Q1 (25th percentile) and Q3 (75th percentile)
Q1 = df['A'].quantile(0.25)
Q3 = df['A'].quantile(0.75)
```

IQR=Q3-Q1

Define bounds for outliers

lower_bound=Q1-1.5*IQR

upper_bound=Q3+1.5*IQR

Eliminate outliers

df_no_outliers=df[(df['A']>=lower_bound)&(df['A']<=upper_bound)]

print("DataFrame after removing outliers:\n", df_no_outliers)

OUTPUT:

DataFrame after removing outliers:

	A
0	1
1	2
2	3
3	4
4	5

EXP.No:03

KNN ALGORITHM FOR CLASSIFICATION

AIM: Apply KNN algorithm for classification and regression

3.a) KNN algorithm for Classification

DESCRIPTION:

train_test_split(): This function in scikit-learn is a fundamental tool for preparing data for machine learning tasks. It divides a dataset into two subsets: a training set and a testing set.

KNeighborsClassifier: It is a supervised machine learning algorithm that implements the k-nearest neighbors (KNN) algorithm for classification tasks. It classifies a new data point based on the majority class of its 'k' nearest neighbors in the training data.

metrics.accuracy_score: It calculates the accuracy of a classification model by finding the proportion of correctly predicted samples.

Classification Report:

It summarizes the performance of a classification model. It includes key metrics such as:

1. **Precision:** Measures the accuracy of positive predictions.
2. **Recall:** Indicates how many actual positives were correctly identified.
3. **F1-Score:** Balances precision and recall into a single score.
4. **Support:** Shows the number of samples for each class.

CONFUSION MATRIX: It is a table that compares the model's predictions against the actual values. The rows represent the actual classes. While the columns represent the predictions we have made. Using this table it is easy to see which predictions are wrong.

The Confusion Matrix has four different quadrants:

	Predicted Yes	Predicted No
Actual Yes	True Positive (TP)	False Negative (FN)
Actual No	False Positive (FP)	True Negative (TN)

Meaning of the above matrix is that, the diagonal values (10, 9, 11) represent the number of samples correctly classified for each of the three Iris species.

PROGRAM:

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn import metrics
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Standardize the features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Create a KNN classifier
knn = KNeighborsClassifier(n_neighbors=5)
# Train the model
knn.fit(X_train, y_train)
# Make predictions
y_pred = knn.predict(X_test)

# Evaluate the model
accuracy = metrics.accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Print the classification report
print("Classification Report:")
print(metrics.classification_report(y_test, y_pred))
# Print the confusion matrix
```

```
print("Confusion Matrix:")
print(metrics.confusion_matrix(y_test,y_pred))
```

OUTPUT:

Accuracy: 1.0

ClassificationReport:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macroavg	1.00	1.00	1.00	30
weightedavg	1.00	1.00	1.00	30

ConfusionMatrix:

```
[[1000]
 [0 90]
 [0 011]]
```

EXP.No:03

KNN ALGORITHM FOR REGRESSION

3. b) KNN algorithm for Regression

DESCRIPTION:

Regression: The term regression is used when we try to find the relationship between variables.

It measures the relationship between the x axis and the y axis, and the value ranges from 0 to 1, where 0 means no relationship, and 1 means totally related.

R²_score(): It measures the relationship between the x axis and the y axis, and the value ranges from 0 to 1, where 0 means no relationship, and 1 means totally related. The sklearn module has a method called **R²_score()** that will help us to find this relationship.

$$R^2 = 1 - \frac{MSE}{Var(y)}$$

Mean Square Error (MSE): It is a statistical metric used to evaluate predictive models. It is a measure of the average squared difference between the actual and predicted values in a regression model. Its value is usually nearer to zero for better models.

$$MSE = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2$$

PROGRAM:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
```

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

#Generate synthetic dataset

```
X,y=make_regression(n_samples=200,n_features=1,noise=0.1,random_state=42)
```

#Split the dataset into training and testing sets

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2, random_state=42)
```

#Create and train the KNN regressor

```
knn_regressor=KNeighborsRegressor(n_neighbors=5)
knn_regressor.fit(X_train, y_train)
```

Make predictions on the test data

```
y_pred=knn_regressor.predict(X_test) #
```

Evaluate the model

```
mse=mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print(f'Mean Squared Error: {mse}')
print(f'R-squared: {r2}')
```

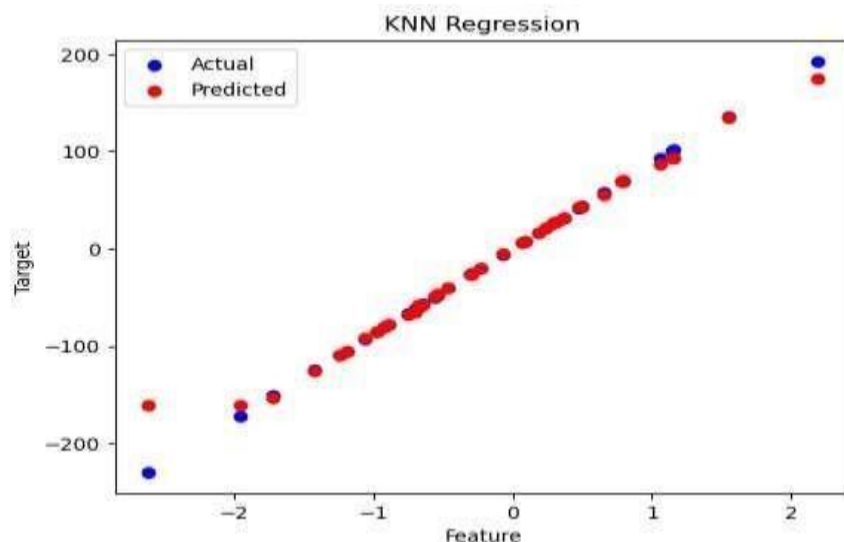
#Visualize the results

```
plt.scatter(X_test, y_test, color='blue', label='Actual')
plt.scatter(X_test,y_pred,color='red',label='Predicted')
plt.title('KNN Regression')
plt.xlabel('Feature')
plt.ylabel('Target')
plt.legend()
plt.show()
```

OUTPUT:

Mean Squared Error (MSE): 133.62045142000457

R²-squared : 0.9817384115764595



EXP.No:04

DECISION TREE FOR CLASSIFICATION

AIM: Demonstrate a ‘Decision Tree’ algorithm for a ‘classification’ problem and perform parameter tuning for better results.

DESCRIPTION:

DECISION TREE:

The Decision Tree classifier operates by recursively splitting the data based on the most informative features.

The working procedure is explained by the following steps

1. Start with the entire dataset at the root node.
2. Select the best feature to split the data (based on measures like Gini impurity).
3. Create child nodes for each possible value of the selected feature.
4. Repeat steps 2–3 for each child node until a stopping criterion is met (e.g., maximum depth reached, minimum samples per leaf, or pure leaf nodes).
5. Assign the majority class to each leaf node

PLOT_TREE:

The „plot_tree“ function is a utility in various machine learning libraries, most notably **scikit-learn**, used to visualize decision trees. It provides a simple way to produce a figure of the tree using matplotlib without needing external dependencies like Graphviz.

GridSearchCV:

It is a tool from „**scikit-learn**“ used to automatically find the best „hyperparameters“ for a machine-learning model. The following three steps describe its working principle as,

1. It tries all combinations of given hyperparameters
2. It evaluates each combination using **cross-validation**
3. It selects the **best-performing model**

PROGRAM:

```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
```

#Step1: Load the Iris dataset

```
iris = load_iris()
```

```
X,y=iris.data,iris.target
```

#Step2: Split the data into training and testing sets

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)
```

#Step3: Train a Decision Tree Classifier with default parameters

```
dt_default = DecisionTreeClassifier(random_state=42)
```

```
dt_default.fit(X_train, y_train)
```

#Predict and evaluate the default model

```
y_pred_default=dt_default.predict(X_test)
```

```
print("Default Model Accuracy:", accuracy_score(y_test, y_pred_default))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_default))
```

```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_default))
```

#Visualize the default decision tree

```
plt.figure(figsize=(12,8))
```

```
plot_tree(dt_default,feature_names=iris.feature_names,class_names=iris.target_names,  
filled=True)
```

```
plt.title("Default Decision Tree")
```

```
plt.show()
```

#Step4: Perform parameter tuning using Grid Search CV

```
param_grid={  
    'criterion':['gini','entropy'],  
    'max_depth':[None,2,3,4,5,6],  
    'min_samples_split':[2,5,10],  
    'min_samples_leaf':[1,2,4]  
}
```

```
grid_search=GridSearchCV(DecisionTreeClassifier(random_state=42),param_grid,cv=5,
```

```
scoring='accuracy', n_jobs=-1)
```

```
grid_search.fit(X_train,y_train)
```

#Best parameters and model

```
print("\nBest Parameters:", grid_search.best_params_)
```

```
best_model = grid_search.best_estimator_
```

#Step5: Evaluate the tuned model

```
y_pred_tuned=best_model.predict(X_test)
```

```
print("\nTuned Model Accuracy:", accuracy_score(y_test, y_pred_tuned))
```

```
print("\nClassification Report:\n", classification_report(y_test, y_pred_tuned))
```

```
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred_tuned))
```

#Visualize the tuned decision tree

```
plt.figure(figsize=(12,8))
```

```
plot_tree(best_model,feature_names=iris.feature_names,class_names=iris.target_names,  
filled=True)
```

```
plt.title("Tuned Decision Tree")
```

```
plt.show()
```

OUTPUT:

Default Model Accuracy: 1.0

Classification Report:

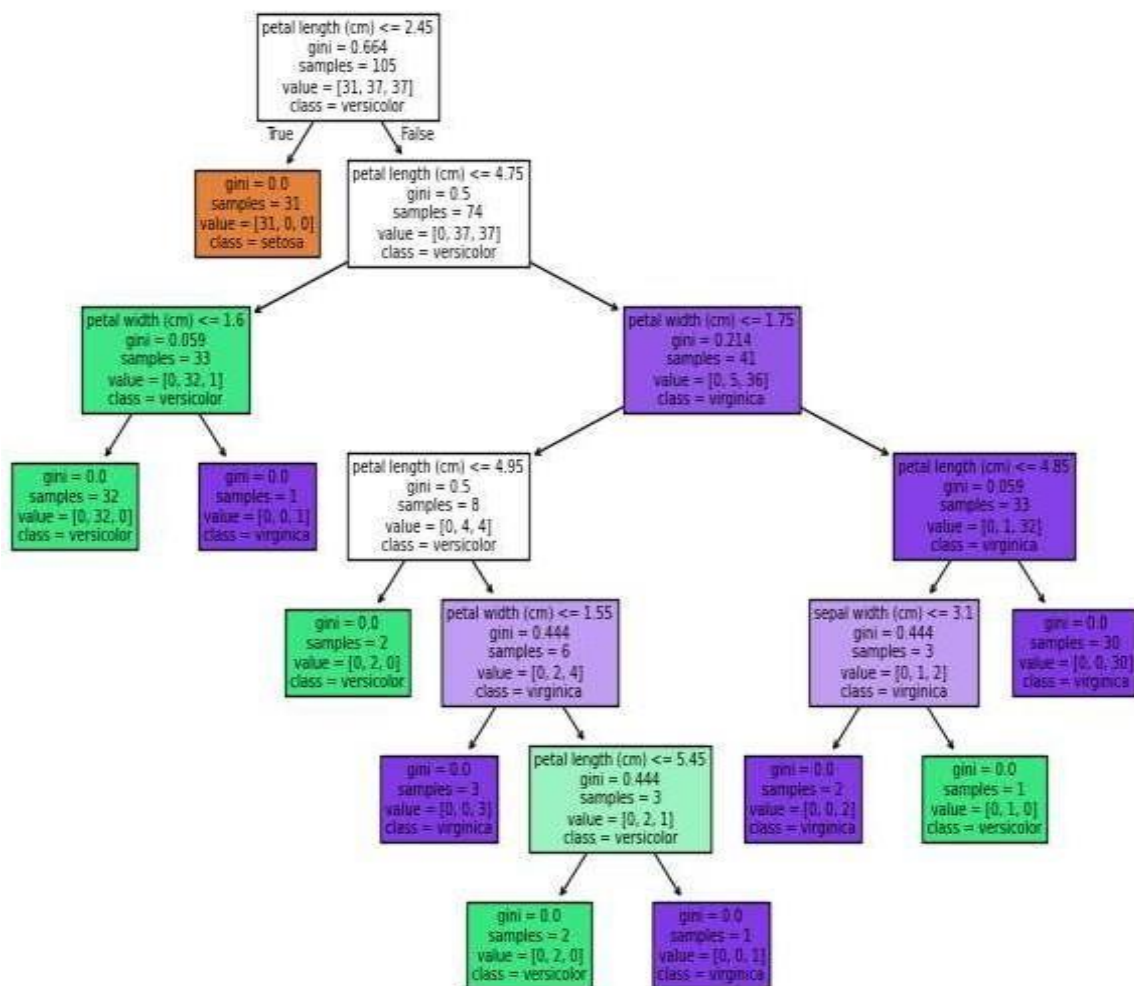
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13

accuracy			1.00	45
macroavg.	1.00	1.00	1.00	45
weightedavg.	1.00	1.00	1.00	45

Confusion Matrix:

```
[[1900]
 [0130]
 [0013]]
```

Default Decision Tree



BEST PARAMETERS:

```
{'criterion':'gini','max_depth':None,'min_samples_leaf':1,'min_samples_split':10}
```

Tuned Model Accuracy: 1.0

Classification Report:

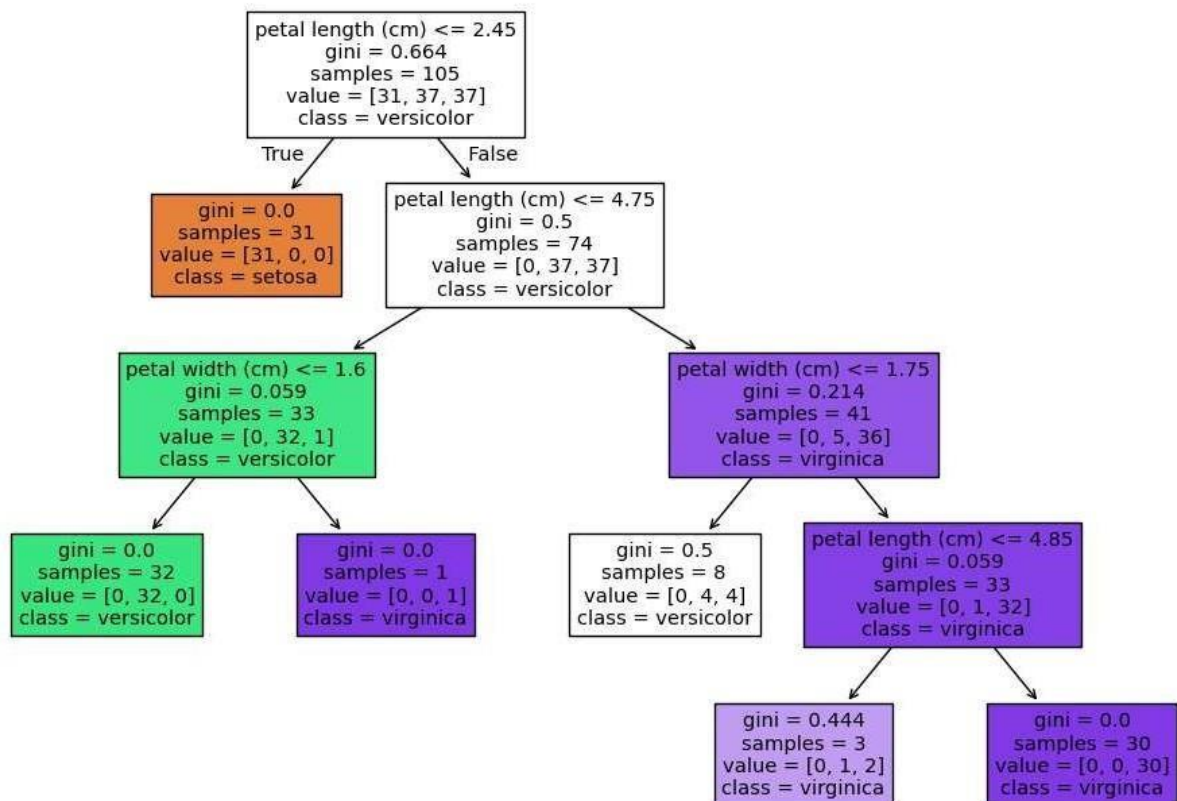
	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13

accuracy			1.00	45
macroavg	1.00	1.00	1.00	45
weightedavg	1.00	1.00	1.00	45

Confusion Matrix:

```
[[1900]
 [0130]
 [0013]]
```

Tuned Decision Tree



DECISION TREE FOR REGRESSION

AIM: Demonstrate ‘Decision Tree’ algorithm for a ‘regression’ problem.

DESCRIPTION:

DATASET: To demonstrate the Decision Tree algorithm, the dataset is needed which is downloaded from a free open source. The dataset is titled or called as „California Housing dataset“. This dataset contains information about houses in California under the specified features or attributes mentioned below.

FEATURES:

- MedInc – Median income
- HouseAge – Average house age
- AveRooms – Average rooms
- AveBedrms – Average bedrooms
- Population
- AveOccup – Average occupancy
- Latitude
- Longitude

TARGET:

- **MedHouseVal** → Median house value (in 100,000s).

DECISION TREE REGRESSOR:

The decision Tree Regressor “is used to analyse the data set network as per the following steps mentioned below,

- Splitting data using feature thresholds.
- Choosing splits that minimize **Mean Squared Error (MSE)**.
- Creating leaf nodes with the **mean target value**.
- Predicting by traversing the tree.

R² SCORE:

The R² score is one of the most important evaluation metrics for regression models. It tells us how well the model explains the variance in the target variable. Higher the value of R² means a better regression model.

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# Step 1: Load the California Housing dataset
```

```
california=fetch_california_housing()
X,y=california.data, california.target
```

#Step2: Split the data into training and testing sets

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=42)
```

#Step3: Train a Decision Tree Regressor with default parameters

```
dt_regressor = DecisionTreeRegressor(random_state=42)
dt_regressor.fit(X_train, y_train)
#Predict on test set
y_pred=dt_regressor.predict(X_test)
```

#Step4: Evaluate the model

```
mse=mean_squared_error(y_test,y_pred)
r2 = r2_score(y_test, y_pred)
```

```
print("Default Decision Tree Regressor Performance:")
print(f'Mean Squared Error (MSE): {mse:.4f}')
print(f'R-squared (R^2): {r2:.4f}')
```

#Step5: Visualize predictions

```
plt.figure(figsize=(10,6))
plt.scatter(y_test,y_pred,alpha=0.7,edgecolor='k')
plt.plot([y_test.min(),y_test.max()], [y_test.min(),y_test.max()], 'r--', lw=2)
plt.xlabel("True Values")
plt.ylabel("Predicted Values")
plt.title("Decision Tree Regression: True vs Predicted")
plt.show()
```

#Perform Grid Search CV for hyperparameter tuning

```
param_grid = {
    'criterion':['squared_error','friedman_mse'],
    'max_depth': [None, 5, 10, 15],
    'min_samples_split':[2,5,10],
    'min_samples_leaf':[1,2,4]
}
grid_search=GridSearchCV(DecisionTreeRegressor(random_state=42),param_grid,cv=5,
scoring='neg_mean_squared_error', n_jobs=-1)
grid_search.fit(X_train,y_train)
```

#Best parameters and model

```
print("\nBest Parameters:",grid_search.best_params_)
best_model = grid_search.best_estimator_
```

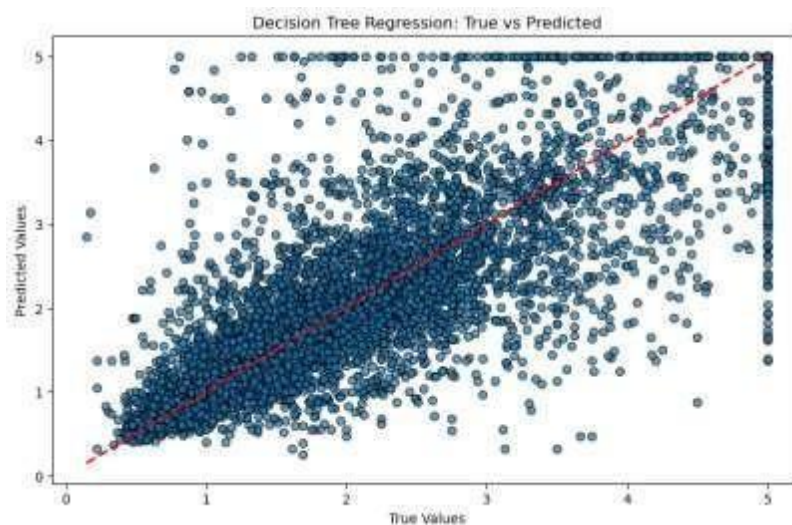
#Evaluate the tuned model

```
y_pred_tuned=best_model.predict(X_test)
```

```
mse_tuned=mean_squared_error(y_test,y_pred_tuned)
r2_tuned = r2_score(y_test, y_pred_tuned)
print("\nTunedDecisionTreeRegressorPerformance:")
print(f'Mean Squared Error (MSE): {mse_tuned:.4f}')
print(f'R-squared (R^2): {r2_tuned:.4f}')
```

OUTPUT:

```
DefaultDecisionTreeRegressorPerformance:
MeanSquaredError(MSE)           :0.5280
R-squared(R2)                   :0.5977
```



BEST PARAMETERS:

```
{'criterion':'squared_error','max_depth':10,'min_samples_leaf':4,'min_samples_split':10}
```

TunedDecisionTreeRegressorPerformance:

```
MeanSquaredError(MSE)           :0.3920
R-squared(R2)                   :0.7014
```

EXP.No:06

RANDOM FOREST ALGORITHM

AIM: Apply Random Forest algorithm for classification and regression

DESCRIPTION:

Let us consider a synthetic dataset called „weather“ with „1000 samples“ to make use of an algorithm, called as „Random Forest“ for „classification“ and „regression“ models of Machine Learning.

- The dataset contains these selected features like Temperature, Humidity, Wind-Speed, Pressure, and Rain-Today where all these features act as input data for ML model.
- „Rain-Tomorrow“ is selected as the target or output feature.

- For the generation of the dataset, a uniform function of random module of NumPy package is used.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score

#Step1: Generate a synthetic weather dataset
np.random.seed(42)
n_samples = 1000
#Weather features
data = {
    'Temperature': np.random.uniform(15, 35, n_samples),
    'Humidity': np.random.uniform(30, 90, n_samples),
    'WindSpeed': np.random.uniform(0, 20, n_samples),
    'Pressure': np.random.uniform(950, 1050, n_samples),
    'RainToday': np.random.choice([0, 1], n_samples, p=[0.7, 0.3]), #0: No Rain, 1: Rain
}

#Target variables
data['RainTomorrow'] = np.where(
    (data['Humidity'] > 70) & (data['RainToday'] == 1), 1, 0 #Simpler rain logic
)
data['TempTomorrow'] = data['Temperature'] + np.random.normal(0, 2, n_samples) #Add noise

#Convert to DataFrame
weather_df = pd.DataFrame(data)

#Step2: Split data for classification
X_class = weather_df[['Temperature', 'Humidity', 'WindSpeed', 'Pressure', 'RainToday']]
y_class = weather_df['RainTomorrow']
X_train_class, X_test_class, y_train_class, y_test_class = train_test_split(X_class, y_class,
test_size=0.3, random_state=42)

#Step3: Random Forest Classifier
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
rf_classifier.fit(X_train_class, y_train_class)
y_pred_class = rf_classifier.predict(X_test_class)
```

#Classification results

```
print("RandomForestClassifier Accuracy:", accuracy_score(y_test_class, y_pred_class))
```

#Step 4: Split data for regression

```
X_reg = weather_df[['Temperature', 'Humidity', 'WindSpeed', 'Pressure', 'RainToday']]
y_reg = weather_df['TempTomorrow']
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg, y_reg, test_size=0.3,
                                                                    random_state=42)
```

#Step 5: Random Forest Regressor

```
rf_regressor = RandomForestRegressor(n_estimators=100, random_state=42)
rf_regressor.fit(X_train_reg, y_train_reg)
y_pred_reg = rf_regressor.predict(X_test_reg)
```

#Regression results

```
mse = mean_squared_error(y_test_reg, y_pred_reg)
r2 = r2_score(y_test_reg, y_pred_reg)
```

```
print("\nRandomForestRegressor Performance:")
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"R-squared (R^2): {r2:.4f}")
```

OUTPUT:

```
RandomForestClassifier Accuracy: 1.0
RandomForestRegressor Performance:
    Mean Squared Error (MSE)           : 4.8041
    R-squared (R^2)                   : 0.8540
```

EXP.No:07

NAÏVE BAYES CLASSIFICATIONALGORITHM

AIM: Demonstrate Naïve Bayes Classification algorithm.

DESCRIPTION:

The „**Titanic dataset**“ is one of the most popular datasets in Machine Learning, mainly used for binary classification problems. It is based on the real Titanic ship disaster (1912). We load it in our program through url link and the objective is to predict **whether a passenger survived or not**.

It contains missing values and has categorical and numerical features. So we need to preprocess it before building a model on it.

Input → Passenger details

Output (Target) → Survived (0 = Not Survived, 1 = Survived)

Common Features (Columns):

FEATURE	DESCRIPTION
Survived	Target variable (0=No, 1=Yes)
Pclass	Passenger class (1st, 2nd, 3rd)
Sex	Male/Female
Age	Age of passenger
SibSp	Siblings/Spouses aboard
Parch	Parents/Children aboard
Fare	Ticket price
Embarked	Port of embarkation (C, Q, S)
Cabin	Cabin number
Ticket	Ticket number
Name	Passenger name

PROGRAM:

```
#Import necessary libraries
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

#Step 1: Load the Titanic dataset

```
data_url =
"https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
titanic_df = pd.read_csv(data_url)
```

#Step 2: Preprocess the dataset and select relevant features and target variable

```
features = ['Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Embarked']
target =
'Survived'
# Drop rows with missing values
titanic_df = titanic_df[features + [target]].dropna()
# Encode categorical variables
titanic_df['Sex'] = titanic_df['Sex'].map({'male': 0, 'female': 1})
titanic_df['Embarked'] = titanic_df['Embarked'].map({'C': 0, 'Q': 1, 'S': 2})
# Display the preprocessed dataset
print("\nPreprocessed Titanic Dataset:\n")
print(titanic_df.head())
# Define features (X) and target (y)
X = titanic_df[features]
Y = titanic_df[target]
```

#Step 3: Split the dataset into training and testing sets

```
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,
random_state=42)
```

#Step4: Train the Gaussian Naïve Bayes model

```
model = GaussianNB()
model.fit(X_train,y_train)
```

#Step5: Predict on the test set

```
y_pred=model.predict(X_test)
```

#Step6: Evaluate the model

```
accuracy=accuracy_score(y_test,y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("\nClassification Report:\n",classification_report(y_test,y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

#Step7: Display some sample predictions

```
sample_predictions=pd.DataFrame({
'Actual': y_test[:10].values,
'Predicted': y_pred[:10]
})
print("\nSample Predictions:\n",sample_predictions)
```

OUTPUT:

Preprocessed Titanic Dataset:

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
0	3	0	22.0	1	0	7.2500	2	0
1	1	1	38.0	1	0	71.2833	0	1
2	3	1	26.0	0	0	7.9250	2	1
3	1	1	35.0	1	0	53.1000	2	1
4	3	0	35.0	0	0	8.0500	2	0

Accuracy: 0.78

Classification Report:

	precision	recall	f1-score	support
0	0.78	0.84	0.81	122
1	0.76	0.70	0.73	92
accuracy			0.78	214
macroavg	0.77	0.77	0.77	214
weightedavg	0.77	0.78	0.77	214

Confusion Matrix:

```
[[10220]
 [2864]]
```

Sample Predictions:

	Actual	Predicted
0	1	1
1	1	1
2	0	1
3	1	1
4	0	0
5	1	1
6	1	1
7	1	1
8	0	0
9	1	0

EXP.No:08

SUPPORT VECTOR ALGORITHM FOR CLASSIFICATION

AIM: Apply Support Vector algorithm for classification

DESCRIPTION:

We create a Titanic dataset with sampled data. After preprocessing the dataset we apply Support Vector algorithm for classification

PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, accuracy_score#

Sample Titanic dataset
data = {
    'PassengerId': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Pclass': [1, 1, 1, 1, 1, 1, 1, 1, 2, 3],
    'Name': [
        'Braund, Mr. Owen Harris', 'Cumings, Mrs. John Bradley (Florence)',
        'Heikkinen, Miss. Laina', 'Futrelle, Mrs. Jacques Heath (Lily May)',
        'Allen, Mr. William Henry', 'Moran, Mr. James', 'McCarthy, Mr. Timothy J',
        'Palsson, Master. Gosta Leonard', 'Johnson, Mrs. Oscar W (Elisabeth Vilhelmina)',
        'Nasser, Mrs. Nicholas (Adele Achem)'],
    'Sex': ['male', 'female', 'female', 'female', 'male', 'male', 'male', 'male', 'female', 'female'],
    'Age': [22, 38, 26, 35, 35, 45, 54, 2, 27, 14],
    'SibSp': [1, 1, 0, 1, 0, 0, 0, 3, 0, 1],
    'Parch': [0, 0, 0, 0, 0, 0, 0, 1, 2, 0],
    'Ticket': ['A/521171', 'PC17599', 'STON/O2.3101282', '113803', '373450', '330877',
               '17463', '349909', '347742', '237736'],
    'Fare': [7.25, 71.2833, 7.925, 53.1, 8.05, 8.4583, 51.8625, 21.075, 11.1333, 30.0708],
```

```

'Cabin':[None,'C85',None,'C123',None,None,'E46',None,None,None], 'Embarked':
['S','C','S','S','S','Q','S','S','S','C'],
'Survived':[0, 1,1, 1,0, 0,0, 1,1, 1]
}
#Create a DataFrame
df=pd.DataFrame(data)
#Preprocessing and Drop the columns that are not useful for the classification task df=
df.drop(['Name', 'Ticket', 'Cabin'], axis=1)
#Fill missing values for 'Age' with the median and 'Embarked' with the mode df['Age']=
df['Age'].fillna(df['Age'].median())
df['Embarked']=df['Embarked'].fillna(df['Embarked'].mode()[0])#
Encode categorical variables: Sex and Embarked
df['Sex']=df['Sex'].map({'male':0,'female':1})
df=pd.get_dummies(df,columns=['Embarked'],drop_first=True)#
Define features (X) and target (y)
X=df.drop('Survived', axis=1)#Features
y=df['Survived'] #Target variable
#Split the dataset into training and testing sets (80% training, 20% testing)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,
random_state=42)

#Feature Scaling: Normalize the features using StandardScaler
scaler = StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test=scaler.transform(X_test)
#Create the SVM model and train it
model=SVC(kernel='linear')#You can also try 'rbf' or 'poly' for different kernels
model.fit(X_train, y_train)
# Make predictions on the test set
y_pred=model.predict(X_test)
#Evaluate the model
accuracy=accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy*100:.2f}%')
print(classification_report(y_test, y_pred))

```

OUTPUT:

```

Accuracy: 100.00%
      precision    recall  f1-score   support

0         1.00      1.00      1.00         1
1         1.00      1.00      1.00         2

accuracy          1.00         3
macroavg          1.00         3
weightedavg       1.00         3

```

EXP.No:09

SIMPLE LINEAR REGRESSIONALGORITHM FOR REGRESSION

AIM: Demonstrates simple linear regression algorithm for a regression problem**DESCRIPTION:**

We build a simple linear regression model to predict the house value based on the number of rooms. For that, we create a dataset which contains the house prices for given number of rooms on which we train the model and predict the output.

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
```

Step1: # Create a simple dataset data

```
={
    'Number_of_Rooms': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10], # Feature
    'House_Price': [100000, 150000, 200000, 250000, 300000, 350000, 400000, 450000,
                    500000, 550000] # Target
}
```

Step2: # Convert the data into a DataFrame df =

```
pd.DataFrame(data)
```

Step3: # Features and target

```
X = df[['Number_of_Rooms']] # Independent variable (Feature)
y = df['House_Price']       # Dependent variable (Target)
```

Step4: # Split the data into training and testing sets (80% training, 20% testing)

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
                                                    random_state=42)
```

Step5: # Create a Linear Regression model

```
model = LinearRegression()
```

Step 6: # Train the model

```
model.fit(X_train, y_train)
```

Step7: # Make predictions on the test set y_pred

```
= model.predict(X_test)
```

Step8: # Evaluate the model

```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

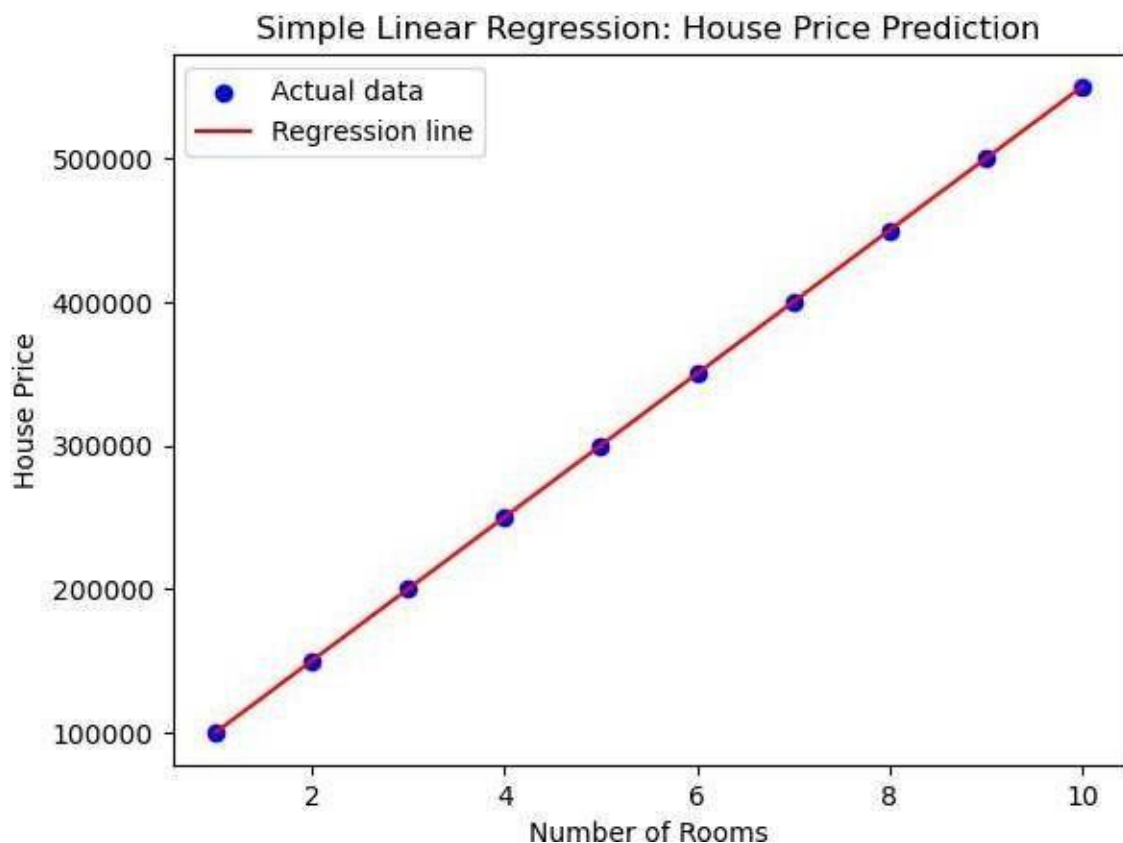
```
print(f'Mean Squared Error: {mse:.2f}')
print(f'R2 Score: {r2:.2f}')
```

Step 9: #Plot the regression line

```
plt.scatter(X, y, color='blue', label='Actual data') # Scatter plot of the data
plt.plot(X, model.predict(X), color='red', label='Regression line')
plt.xlabel('Number of Rooms')
plt.ylabel('House Price')
plt.title('Simple Linear Regression: House Price Prediction')
plt.legend()
plt.show()
```

OUTPUT:

```
Mean Squared Error      : 0.00
R2 Score                 : 1.00
```



EXP.No:10

LOGISTIC REGRESSION ALGORITHM FOR A CLASSIFICATION

AIM: Apply Logistic regression algorithm for a classification problem

DESCRIPTION:

We built a Logistic Regression model for Heart Disease Prediction using a synthetic dataset and we use ROC curve to plot logistic regression

RECEIVER OPERATING CHARACTERISTIC (ROC):

ROC stands for Receiver Operating Characteristic. This **ROC** is one of the most important evaluation tools for classification models like Logistic Regression. It is a curve that shows how well a classifier separates two classes by plotting, True Positive Rate Vs False Positive Rate for different classification thresholds.

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

# Generates synthetic dataset for heart disease prediction
np.random.seed(42)

# Number of samples
num_samples = 1000

# Features
age = np.random.randint(20, 80, size=num_samples) # Age between 20 and 80
sex = np.random.choice([0, 1], size=num_samples) # 0 = female, 1 = male
cp = np.random.choice([1, 2, 3, 4], size=num_samples) # Chest pain type
trestbps = np.random.randint(90, 200, size=num_samples) # Resting blood pressure in mmHg
chol = np.random.randint(150, 400, size=num_samples) # Serum cholesterol in mg/dl
fbs = np.random.choice([0, 1], size=num_samples) # Fasting blood sugar >120 mg/dl
restecg = np.random.choice([0, 1, 2], size=num_samples) # Resting electrocardiographic results
thalach = np.random.randint(60, 200, size=num_samples) # Maximum heart rate achieved
exang = np.random.choice([0, 1], size=num_samples) # Exercise induced angina
oldpeak = np.random.uniform(0, 6, size=num_samples) # ST depression induced by exercise relative to rest
slope = np.random.choice([1, 2, 3], size=num_samples) # Slope of the peak exercise ST segment
ca = np.random.choice([0, 1, 2, 3], size=num_samples) # Number of major vessels colored by fluoroscopy
thal = np.random.choice([1, 2, 3], size=num_samples) # Thalassemia
target = np.random.choice([0, 1], size=num_samples) # 0 = no heart disease, 1 = heart disease
```

Step1: # Create DataFrame from synthetic data df =

```
pd.DataFrame({
    'age': age,
    'sex': sex,
    'cp': cp,
    'trestbps': trestbps,
    'chol': chol,
    'fbs': fbs, 'restecg':
    restecg,
    'thalach': thalach,
    'exang': exang,
    'oldpeak': oldpeak,
    'slope': slope,
    'ca': ca,
    'thal': thal,
    'target': target
})
```

Step2: # Define the features (X) and target variable (y)

```
X = df.drop('target', axis=1) # 'target' is the column we're predicting
y = df['target'] # Binary outcome: 0 = no heart disease, 1 = heart disease
```

Step3: # Split the data into training and testing sets (80% training, 20% testing) X_train,

```
X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Step4: # Feature Scaling (Normalize the features) scaler =

```
StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 5: # Create the Logistic Regression model

```
model = LogisticRegression(max_iter=200)
```

Step 6: # Train the model

```
model.fit(X_train, y_train)
```

Step7: # Make predictions on the test set y_pred

```
= model.predict(X_test)
```

Step8: # Evaluate the model

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")
```

Step9: # Prepare Confusion Matrix

```
conf_matrix = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(conf_matrix)
```

Step10: # Compute Classification Report

```
class_report = classification_report(y_test, y_pred)
```

```
print("Classification Report:")
```

```
print(class_report)
```

Step 11: #Plot ROC curve

```
y_prob=model.predict_proba(X_test)[:,-1]fpr,
tpr, _ = roc_curve(y_test, y_prob) roc_auc =
auc(fpr, tpr)
plt.plot(fpr,tpr,label=f' AUC={roc_auc:.2f}')
plt.plot([0,1], [0,1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROCCurve-LogisticRegression")
plt.legend()
plt.show()
```

OUTPUT:

Accuracy: 49.00%

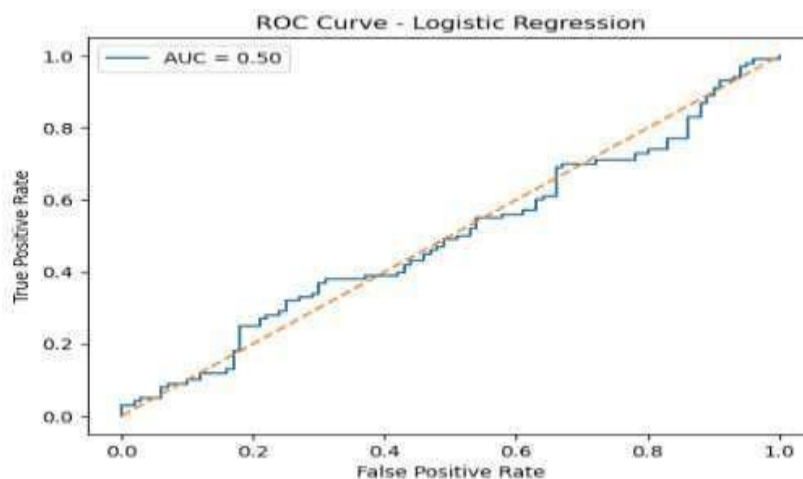
Confusion Matrix:

```
[[ 5248
```

```
 [ 5446]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.49	0.52	0.50	100
1	0.49	0.46	0.47	100
accuracy			0.49	200
macroavg	0.49	0.49	0.49	200
weightedavg	0.49	0.49	0.49	200



EXP.No:11

MULTI-LAYER PERCEPTRON ALGORITHM FOR A CLASSIFICATION

AIM: Demonstrate Multi-layer Perceptron algorithm for a classification problem

DESCRIPTION:

We built a Multi-layer Perceptron model for Heart Disease Prediction using a synthetic dataset. The dataset contains patient medical attributes used to predict the presence of heart

disease. Each **row** of dataset represents one patient. Each **column** represents a medical feature. The **target** variable indicates whether the patient has heart disease or not.

MLPClassifier():

It is available in **scikit-learn** library and used to implement a **Multi-Layer Perceptron (MLP)** for classification problems. It uses **backpropagation** to train the network.

Its syntax is

```
MLPClassifier(
    hidden_layer_sizes=(100,),
    activation='relu',
    solver='adam',
    max_iter=200,
    learning_rate_init=0.001,
    random_state=None
)
```

Parameter	Meaning
hidden_layer_sizes	Tuple representing number of neurons in hidden layers
activation	Activation function (relu, tanh, logistic)
solver	Optimization algorithm (adam, sgd, lbfgs)
max_iter	Maximum number of training iterations
learning_rate_init	Initial learning rate
random_state	Controls randomness

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Generates synthetic dataset for heart disease prediction
np.random.seed(42)

# Number of samples
num_samples = 1000

# Features
age = np.random.randint(20, 80, size=num_samples) # Age between 20 and 80
sex = np.random.choice([0, 1], size=num_samples) # 0 = female, 1 = male
cp = np.random.choice([1, 2, 3, 4], size=num_samples) # Chest pain type
trestbps = np.random.randint(90, 200, size=num_samples) # Resting blood pressure
```

(Approved by AICTE, New Delhi, Affiliated to JNTUK, Kakinada, Accredited with „A“ grade by NAAC, Accredited by NBA (Tier-I))

```

chol=np.random.randint(150,400,size=num_samples)#Serumcholesterol
fbs=np.random.choice([0,1],size=num_samples)#Fastingbloodsugar>120
mg/dl
restecg=np.random.choice([0,1,2],size=num_samples)#Resting
electrocardiographicresults
thalach=np.random.randint(60,200,size=num_samples)#Maximumheartrate
achieved
exang=np.random.choice([0,1],size=num_samples)#Exerciseinducedangina
oldpeak=np.random.uniform(0,6,size=num_samples)#STdepressioninduced    by
exercise relative to rest
slope=np.random.choice([1,2,3],size=num_samples)#Slopeofthepeak
exerciseSTsegment
ca=np.random.choice([0,1,2,3],size=num_samples)#Numberofmajor
vessels colored by fluoroscopy
thal=np.random.choice([1,2, 3],size=num_samples)# Thalassemia
target=np.random.choice([0,1],size=num_samples)#0=noheartdisease,1=    heart
disease

#CreateDataFramefromsyntheticdatadf
=pd.DataFrame({
    'age': age,
    'sex':sex,
    'cp':cp,
    'trestbps':trestbps,
    'chol': chol,
    'fbs':fbs,'restecg':
restecg, 'thalach':
thalach, 'exang':
exang,
    'oldpeak':oldpeak,
    'slope': slope,
    'ca':ca,
    'thal': thal,
    'target':target
})
#Definethefeatures(X)andtargetvariable(y)
X=df.drop('target',axis=1)# 'target'isthe columnwe're predicting
y=df['target']#Binaryoutcome:0=noheartdisease,1=heartdisease

#Split the data into training and testing sets(80%training,20%testing)
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,
random_state=42)
#FeatureScaling(Normalizethe features)

```

```

scaler= StandardScaler()
X_train=scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
#Create the Multi-layer Perceptron (MLP) Classifier model
mlp=MLPClassifier(hidden_layer_sizes=(50,50),max_iter=1000,
random_state=42)

#Train the model
mlp.fit(X_train,
y_train)#Make predictions on the test set
y_pred=mlp.predict(X_test)#
Evaluate the model
accuracy=accuracy_score(y_test,y_pred)
print(f"Accuracy: {accuracy*100:.2f}%")

#Confusion Matrix
conf_matrix=confusion_matrix(y_test,y_pred)
print("Confusion Matrix:")
print(conf_matrix)

# Classification Report
class_report=classification_report(y_test,y_pred)
print("Classification Report:")print(class_report)

```

OUTPUT:

```

Accuracy: 48.50%
Confusion Matrix:
[[48 52]
 [51 49]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.48	0.48	0.48	100
1	0.49	0.49	0.49	100
accuracy			0.48	200
macroavg	0.48	0.48	0.48	200
weightedavg	0.48	0.48	0.48	200

EXP.NO:12

K-MEANS CLUSTERING ALGORITHM

AIM:

Implement the K-means algorithm and apply it to the data you selected. Evaluate performance by measuring the sum of the Euclidean distance of each example from its class center. Test the performance of the algorithm as a function of the parameters K.

DESCRIPTION:

We implement K-means algorithm for Heart Disease dataset. The dataset contains patient medical attributes used to predict the presence of heart disease. Each **row** of dataset represents one patient. Each **column** represents a medical feature. The **target** variable is used for clustering.

KMeans(): It is a clustering algorithm available in **scikit-learn library** and used for unsupervised learning. It partitions data into **K clusters** such that:

- Each data point belongs to the cluster with the nearest mean (centroid).
- It minimizes **within-cluster variance**.

Its syntax is

```
KMeans(
    n_clusters=8,
    init='k-means++',
    max_iter=300,
    n_init=10,
    random_state=None
)
```

Parameter	Meaning
n_clusters	Number of clusters (K)
init	Method to initialize centroids (k-means++ recommended)
max_iter	Maximum iterations
n_init	Number of times algorithm runs with different centroid seeds
random_state	Controls randomness

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import pairwise_distances_argmin_min
```

#Generate synthetic dataset for heart disease prediction np.random.seed(42)

#Number of samples

```
num_samples=1000#
```

Features

```
age=np.random.randint(20,80,size=num_samples)#Age between 20 and 80
sex=np.random.choice([0, 1], size=num_samples)# 0 = female, 1 = male
cp=np.random.choice([1, 2, 3, 4], size=num_samples)# Chest pain type
trestbps=np.random.randint(90,200,size=num_samples)#Resting blood pressure
chol=np.random.randint(150, 400, size=num_samples)# Serum cholesterol
fbs=np.random.choice([0,1],size=num_samples)#Fasting blood sugar >120mg/dl
restecg=np.random.choice([0,1,2],size=num_samples)#Resting electrocardiographic
```

```

results
thalach=np.random.randint(60,200,size=num_samples)#Maximumheartrateachieved exang
= np.random.choice([0, 1], size=num_samples)# Exercise induced angina
oldpeak=np.random.uniform(0,6,size=num_samples)#STdepressioninducedby
                                exerciserelativetorest
slope=np.random.choice([1,2,3],size=num_samples)#Slopeofthepeakexercise
                                ST segment
ca=np.random.choice([0,1,2,3],size=num_samples)#Numberofmajorvessels
                                coloredbyfluoroscopy
thal = np.random.choice([1, 2, 3], size=num_samples)# Thalassemia
target=np.random.choice([0,1], size=num_samples)# 0=no heartdisease,1 =
                                heart disease

#CreateDataFramefromsyntheticdatadf
    =pd.DataFrame({
        'age': age,
        'sex':sex,
        'cp':cp,
        'trestbps':trestbps,
        'chol': chol,
        'fbs': fbs,
        'restecg':restecg,
        'thalach':thalach,
        'exang': exang,
        'oldpeak':oldpeak,
        'slope': slope,
        'ca':ca,
        'thal': thal,
        'target':target
    })

#Definethe features (X) andtargetvariable (y) (weuseonlythefeaturesfor clustering)
    X=df.drop('target',axis=1)#Removethetargetcolumnforclustering
    y=df['target']#Thisisnotusedforclusteringbutcanbeusefulforanalysislater#Feature
Scaling(Normalizethefeatures)
    scaler =
    StandardScaler()X_scaled=scaler.f
    it_transform(X)

#FunctiontoevaluateK-meansfordifferentvaluesofKdef
    evaluate_kmeans(X, K_values):
        for K in K_values:
            print(f"\nEvaluatingK={K}...")

# ApplyK-meansclustering
    kmeans=KMeans(n_clusters=K,random_state=42)
    kmeans.fit(X)

```

```
# Calculate the sum of Euclidean distances of each sample to its cluster center
distances=pairwise_distances_argmin_min(X,kmeans.cluster_centers_)[1]
```

```
sum_of_distances = np.sum(distances)
```

```
#Print the results
```

```
print(f"Sum of Euclidean distances for K={K}:{sum_of_distances:.2f}")#
```

Optionally, you can print the cluster centers to inspect

```
print("Cluster centers:\n",kmeans.cluster_centers_)#
```

Test performance for different values of K

```
K_values=[2,3,4,5]#Number of clusters to test
```

```
evaluate_kmeans(X_scaled, K_values)
```

OUTPUT:

Evaluating K=2...

Sum of Euclidean distances for K=2: 3441.96 Cluster centers:

```
[ [1.55448890e-02 -5.62580905e-02 -1.52669586e-03 1.00526910e-02
 1.34553519e-02 8.72354940e-03 -3.86163597e-02 -3.73082281e-03
 9.37922837e-02 12.88313463e-02 -1.14781582e-02 -3.42278303e-03
 -3.75481421e-04]
 [-1.76706858e-02 6.39515046e-02 1.73547478e-03 -1.14274179e-02
 -1.52954000e-02 -9.91651342e-03 4.38972294e-02 4.24102080e-03
 -1.06618579e+00 -3.27740945e-02 1.30478209e-02 3.89085593e-03
 4.26829307e-04]]
```

Evaluating K=3...

Sum of Euclidean distances for K=3: 3355.53 Cluster centers:

```
[ [9.89971708e-03 1.03459817e+00 1.22280768e-01 8.80929862e-02
 -4.82365665e-02 1.43995744e-01 -8.39350352e-02 -1.09788329e-01
 9.37922837e-02 13.09759937e-02 -1.06338970e-01 -1.17071826e-01
 -1.26311743e-01]
 [2.02556877e-02 -9.66558833e-01 -1.04841890e-01 -5.50705899e-02
 6.49361941e-02 -1.04158765e-01 -7.98706311e-02 48.47723376e-02
 9.37922837e-02 12.70416750e-02 6.76815537e-02 9.14153837e-02
 1.04716158e-01]
 [-1.76706858e-02 6.39515046e-02 1.73547478e-03 -1.14274179e-02
 -1.52954000e-02 -9.91651342e-03 4.38972294e-02 4.24102080e-03
 -1.06618579e+00 -3.27740945e-02 1.30478209e-02 3.89085593e-03
 4.26829307e-04]]
```

Evaluating K=4...

Sum of Euclidean distances for K=4: 3328.22 Cluster centers:

```
[ [0.42230756 -0.09380754 -0.12468801 0.3913122 -0.04063948 0.42977759
 -0.12081762 -0.17568372 0.74369205 0.00257795 0.10779139 0.79159005
 -0.00575132]
 [-0.04391215 -0.68306159 -0.13670289 -0.48526066 -0.10490275 -0.68013604
 0.35739305 0.17883361 0.45359659 0.12789475 0.34437717 -0.37861461
 0.14981709]
 [-0.12848307 0.33652015 -0.06637182 -0.40825944 0.05240311 0.15181331
```

```
0.10471786-0.09315157 -0.97297143 -0.53505537 0.12123475 0.44540453
0.07239224]
[-0.189731890.35784326 0.28449746 0.48351136 0.07593389 0.09638291
-0.3104216 0.07633906-0.096925980.388234 -0.50326448-0.74086474
-0.19391909]]
```

Evaluating K=5...

Sum of Euclidean distances for K=5: 3267.32 Cluster centers:

```
[ [0.16818968-0.83395204-0.393547240.332413480.103218680.63458476
-0.263100540.170202190.21354623-0.09492591-0.031010790.89787774
0.1884962]
[0.06849852-0.754898 -0.00397332 -0.40254246 -0.06407473 -0.62435565
0.4203006 0.18777714 0.55251733 -0.00722075 0.42375405 -0.34657227
0.32817076]
[0.238872060.59956404 -0.05684176 -0.5823881 -0.18281077 -0.24091775
0.39074102-0.00450207 -0.82656411 -0.48586802 0.08790899 0.64287189
-0.05309253]
[-0.19050565 0.04811232 -0.08470056 0.35590485 -0.22497763 -0.49183547
-0.45410944 0.26991516 -0.33228686 0.52431112 -0.63849778 -0.57877046
-0.22028772]
[-0.19887308 0.76369919 0.41334235 0.26157148 0.33952253 0.75814291
-0.08261516-0.541374490.31656165-0.021916750.16083722-0.31428381
-0.18715973]]
```

EXP.NO:13

FUZZY C-MEANS CLUSTERING ALGORITHM

AIM: Demonstrate the use of Fuzzy C-Means Clustering

DESCRIPTION:

We implement C-means clustering algorithm for Heart Disease dataset. The dataset contains patient medical attributes used to predict the presence of heart disease. Each **row** of dataset represents one patient. Each **column** represents a medical feature. The **target** variable is used for clustering.

Summary of command to work above we need to install fuzzy in anaconda-prompt

```
conda activate myenv # If you're using a virtual environment, activate it
pip install scikit-fuzzy
```

after installation check same cmd to see if it is successful or not

```
Python import
skfuzzy as fuzz
print("skfuzzy installed successfully!")
```

skfuzzy: It is a Python library used for **Fuzzy Logic** and **Fuzzy Clustering** operations. It is part of the **Scikit-Fuzzy** package and is commonly used in Machine Learning, control systems, and decision-making applications.

Skfuzzy.cmeans: It is a function in the **Scikit-Fuzzy (skfuzzy)** library used to perform **Fuzzy C-Means (FCM) clustering**. Its syntax is

```
skfuzzy.cmeans(
    data, c, m, error, maxiter,
```

```
metric='euclidean',
init=None,
seed=None
```

)

Parameter	Description
data	Input dataset in matrix form
c	Number of clusters
m	Fuzziness coefficient ((m>1))
error	Stopping criterion
maxiter	Maximum number of iterations
metric	Distance metric (default: Euclidean)
init	Initial membership matrix
seed	Random seed value

PROGRAM:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
import skfuzzy as fuzz
import matplotlib.pyplot as plt

# Generates synthetic dataset for heart disease prediction
np.random.seed(42)
# Number of samples
num_samples = 1000
# Features
age = np.random.randint(20, 80, size=num_samples) # Age between 20 and 80
sex = np.random.choice([0, 1], size=num_samples) # 0 = female, 1 = male
cp = np.random.choice([1, 2, 3, 4], size=num_samples) # Chest pain type
trestbps = np.random.randint(90, 200, size=num_samples) # Resting blood pressure
chol = np.random.randint(150, 400, size=num_samples) # Serum cholesterol
fbs = np.random.choice([0, 1], size=num_samples) # Fasting blood sugar > 120mg/dl
restecg = np.random.choice([0, 1, 2], size=num_samples) # Resting electrocardiographic results
thalach = np.random.randint(60, 200, size=num_samples) # Maximum heart rate achieved
exang = np.random.choice([0, 1], size=num_samples) # Exercise induced angina
oldpeak = np.random.uniform(0, 6, size=num_samples) # ST depression induced by exercise relative to rest
slope = np.random.choice([1, 2, 3], size=num_samples) # Slope of the peak exercise ST segment
```

```

ca=np.random.choice([0,1,2,3],size=num_samples)#Number of major vessels colored by
fluoroscopy
thal=np.random.choice([1,2,3],size=num_samples)#Thalassemia

#Create DataFrame from synthetic data df
df=pd.DataFrame({
    'age': age,
    'sex': sex,
    'cp': cp,
    'trestbps': trestbps,
    'chol': chol,
    'fbs': fbs,
    'restecg': restecg,
    'thalach': thalach,
    'exang': exang,
    'oldpeak': oldpeak,
    'slope': slope,
    'ca': ca,
    'thal': thal
})

# Save the synthetic dataset to a CSV file
df.to_csv('synthetic_heart_disease_fcm.csv',index=False)
print("Synthetic CSV file 'synthetic_heart_disease_fcm.csv' created successfully!")

#Load the dataset from the CSV file (for demonstration) df=
pd.read_csv('synthetic_heart_disease_fcm.csv')

#Display the first few rows of the dataset
print(df.head())

#Define the features (X) for clustering
X=df.drop('age',axis=1)#Remove 'age' for simplicity in clustering

#Feature Scaling (Normalize the features) scaler
=StandardScaler()
X_scaled=scaler.fit_transform(X)

#Apply Fuzzy C-Means Clustering using skfuzzy
n_clusters = 3# Define the number of clusters
cntr,u,u0,d,jm,p,fpc=fuzz.cmeans(X_scaled.T,c=n_clusters,m=2,error=0.005,maxiter=1000)

```

```
#The matrix `u` contains the membership values for each sample to each cluster #Get
the cluster centers from `cntr`
cluster_centers = cntr
```

```
#Predict the cluster each point belongs to (based on highest membership)
predictions = np.argmax(u, axis=0)
```

```
#Print the results
print("\n Fuzzy C-Means Membership Values (First 5 samples):")
print(u[:, :5]) #Show the first 5 membership values
```

```
# Print cluster centers
print("\n Cluster Centers:")
print(cluster_centers)
```

```
#Plot membership values for the first 5 samples
plt.figure(figsize=(10,6))
for i in range(n_clusters): #For each cluster
    plt.plot(u[i, :], label=f'Cluster {i+1}')
```

```
plt.title("Fuzzy Membership Values for Each Cluster")
plt.xlabel("Sample Index")
plt.ylabel("Membership Degree")
plt.legend()
plt.show()
```

OUTPUT:

Synthetic CSV file 'synthetic_heart_disease_fcm.csv' created successfully!

```
agesexcp trestbpschol fbsrestecgthalachexangoldpeak\
058    1 1    173 305  0    1    170    10.044118
171    1 2    157 251  1    2    188    14.150321
248    0 4     96 399  0    1    124    13.968056
334    1 2    143 372  0    1    138     00.230405
462    0 2     90 208  1    0     81    12.209501
```

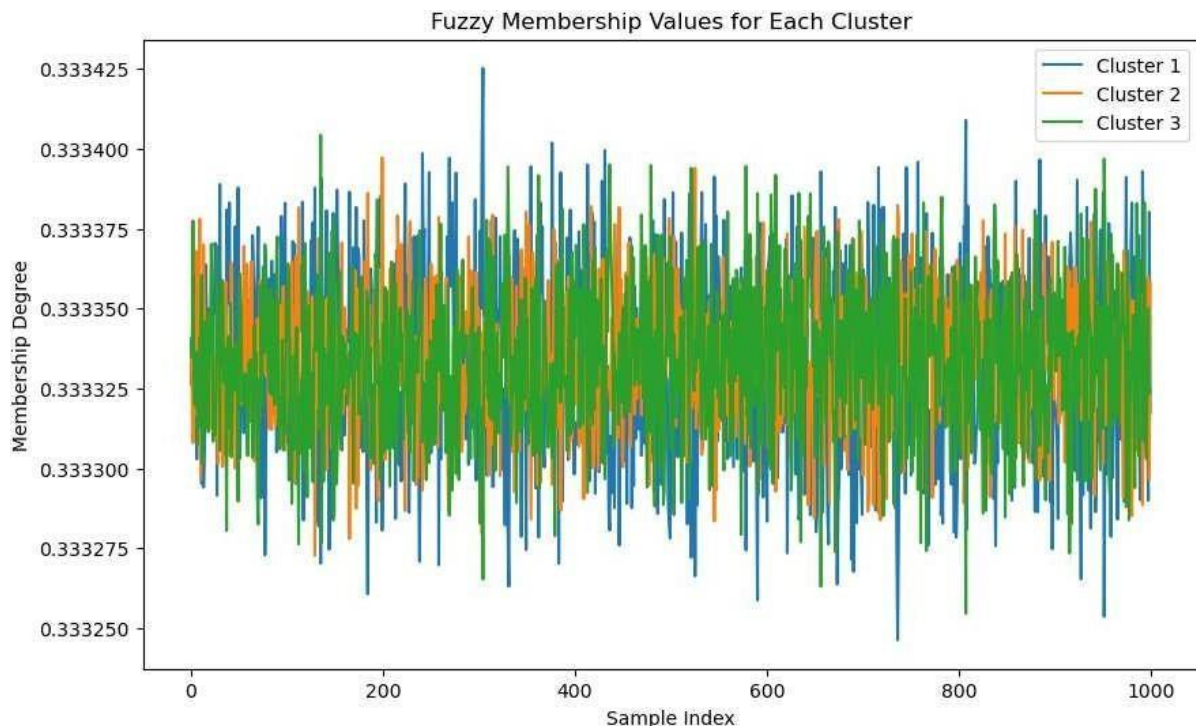
```
      slope  ca  thal
0      3    1    1
1      3    0    3
2      3    1    3
3      3    2    1
4      3    1    3
```

Fuzzy C-Means Membership Values (First 5 samples):

```
[[0.333326480.33334117 0.33331460.333360450.33332587]
[0.33333420.33332390.333307890.333312820.33335455]
[0.333339320.333334930.333377520.333326730.33331958]]
```

Cluster Centers:

```
[[6.28722104e-05-6.50476962e-061.46044085e-061.53124510e-04
 3.64354619e-049.91222822e-05-1.67342562e-05-1.50385943e-04
-1.07834567e-051.09589835e-041.34116186e-04-1.26408199e-04]
[6.29537901e-05-1.53057191e-04-7.08602809e-05-2.15690680e-04
-8.92064919e-05-1.53495394e-04-3.74472637e-052.99846322e-05
-2.97261341e-06-1.19593749e-04-4.92875419e-053.86966860e-05][[-
1.25824961e-041.59565634e-046.93940243e-056.25669296e-05
-2.75143297e-045.43733119e-055.41841997e-051.20402041e-04
1.37511456e-051.00006828e-05-8.48255647e-058.77140396e-05]]
```



Exp.No:14

EXPECTATION MAXIMIZATION BASED CLUSTERING ALGORITHM

AIM: Demonstrate the use of Expectation Maximization based clustering algorithm

DESCRIPTION:

Generate a synthetic dataset containing three two-dimensional clusters for use in clustering or machine learning experiments.

Cluster1: cluster1=np.random.randn(300,2)+np.array([2,2])

- 300 data points

- each with 2 features (x and y coordinates)
- values drawn from a standard normal distribution.
- Adding $[2, 2]$ shifts the cluster center to approximately: So this forms the first cluster around point (2, 2)

Cluster2: `cluster2=np.random.randn(300,2)+np.array([-5,-5])`

- Creates another 300 two-dimensional points.
- Shifts them to center around: $(-5, -5)$

Cluster3: `cluster3=np.random.randn(300,2)+np.array([7,-3])`

- Generates the third set of 300 points.
- Cluster center becomes: $(7, -3)$

Now all 3 clusters are combined into a single dataset for machine learning experiments as `X=np.vstack([cluster1, cluster2, cluster3])`

GaussianMixture(): It is a method of sklearn.mixture module used to implement the Gaussian Mixture Model (GMM) for probabilistic clustering. Its syntax is

```
GaussianMixture(
    n_components=1,
    covariance_type='full',
    tol=0.001,
    max_iter=100,
    random_state=None
)
```

Parameter	Description
n_components	Number of Gaussian clusters
covariance_type	Shape of covariance matrix
tol	Convergence threshold
max_iter	Maximum EM iterations
random_state	Seed for reproducibility

PROGRAM:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

# Generates synthetic data (two-dimensional data with 3 clusters)
np.random.seed(42)
# Cluster 1 (center at (2, 2))
cluster1 = np.random.randn(300, 2) + np.array([2, 2]) #
Cluster 2 (center at (-5, -5))
cluster2 = np.random.randn(300, 2) + np.array([-5, -5]) #
Cluster 3 (center at (7, -3))
cluster3 = np.random.randn(300, 2) + np.array([7, -3]) #
Combine all clusters to form the dataset
X = np.vstack([cluster1, cluster2, cluster3])
```

```
#Convert to DataFrame for easier handling
df=pd.DataFrame(X,columns=["Feature1", "Feature2"])

#Plot the synthetic data before clustering
plt.figure(figsize=(8, 6))
plt.scatter(df['Feature1'],df['Feature2'],s=10,color='gray',alpha=0.5)
plt.title("Synthetic Data before Clustering")
plt.xlabel('Feature1')
plt.ylabel('Feature2')
plt.show()

#Normalize the features (optional but recommended for better clustering) scaler
=StandardScaler()
X_scaled=scaler.fit_transform(X)

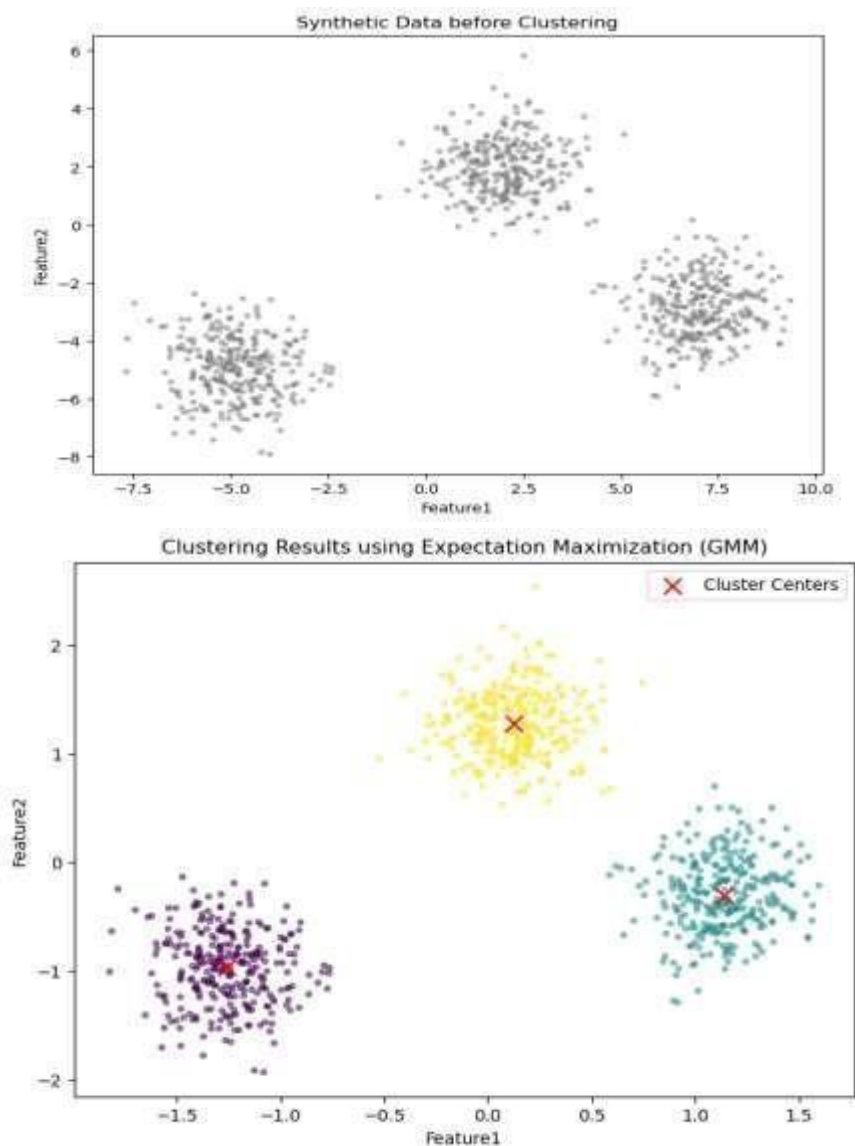
#Apply Gaussian Mixture Model (GMM) using Expectation Maximization
gmm=GaussianMixture(n_components=3,covariance_type='full',random_state=42)
gmm.fit(X_scaled)

#Predict the cluster for each data point
predictions=gmm.predict(X_scaled)

# Get the cluster centers
cluster_centers=gmm.means_#
Plot the clustered data
plt.figure(figsize=(8, 6))
plt.scatter(X_scaled[:,0],X_scaled[:,1],c=predictions,s=10,cmap='viridis',alpha=0.5)
plt.scatter(cluster_centers[:, 0], cluster_centers[:, 1], c='red', marker='x', s=100,
label='Cluster Centers')
plt.title("Clustering Results using Expectation Maximization (GMM)")
plt.xlabel('Feature1')
plt.ylabel('Feature2')
plt.legend()
plt.show()

#Print cluster centers
print("Cluster Centers(after EM algorithm):")
print(cluster_centers)
# Evaluate the results (optional)
print("\nPredicted Labels for First 10 Samples:")
print(predictions[:10])
```

OUTPUT:



ClusterCenters(afterEMalgorithm):

$\begin{bmatrix} -1.25915483 & -0.97010161 \end{bmatrix}$

$\begin{bmatrix} 1.13551144 & -0.30772272 \end{bmatrix}$

$\begin{bmatrix} 0.123590421 & 2.7790734 \end{bmatrix}$

PredictedLabelsforFirst10Samples:[2

22222222]